

基于连续覆盖证书与观测共享调度的无线电干扰源自动定位与清除

摘 要

针对干扰源个数、位置、接收半径及定向方向未知，且测角误差有界、同地点误差固定的自动清除任务，建立以相容位置集合为信息状态、以实际任务时间为目标的搜索定位模型。将“如何证明不遗漏”与“如何减少移动和检测费用”分别处理，再通过证据账本与预算检查连接，形成可在线执行的完整策略。

针对问题一，将示向度转化为前向楔形的半平面约束，先判定交会区域的空集、无界及有界情形，再通过凸包和旋转卡壳计算直径。构造可由三次测向产生的正三角形反例，证明以区域直径为直径的圆不一定覆盖区域，并给出最远点对中点圆的充要检验条件。

针对问题二，利用“首次已接收”建立源距离与未知接收半径的耦合约束，推导三个圆盘与前向半平面的交作为保证接收候选域。对两条误差带交集的解析直径上界进行有界优化：当目标上界为 163 米时，得到移动距离 886.3434 米、计算下界 886.2614 米，间隙 0.08194 米；该最优性仅针对所用解析判据。

针对问题三，采用原点与半径 1125 米六点环构成的七站布局。联合维护未知、已发现、已清除及已排除频道，把剩余搜索站和已知源放入同一服务任务排序。在局部定位中复用完整观测历史，在可清除时直接投影到最近的保证清除位置，并通过有限试探额度与下降秩保证备用过程可接续执行。

针对问题四，采用 21 站布局，并以“合法源位置位于 1000 米内测点凸包中”为定向发现的充分条件。以连续区域证书核查覆盖；用真实正、负观测估计补测位置的接收概率，选择共享观察点与原地补测。对狭窄可行域构造完整光学覆盖链，通过 Voronoi 分区逐块证明覆盖后，按预计首次成功成本安排清除次序。

30 场离线合成案例中，390 个源全部清除。问题三、四各 10 场常规输入的平均总时间分别为 3060.13 秒和 6261.69 秒，移动时间分别占约 76.01% 和 74.01%。文中区分离线结果、截图报告的演练成绩与尚未完成的正式测试，正式测试及未测敏感性数据保留空栏。

关键词：有界误差；交会定位；连续覆盖；观测共享；联合调度；光学清除

1 问题重述

1.1 背景与任务边界

无线电干扰源的初步监测只能确定目标分布区域，精准定位仍需要近距离观测和现场清除。题目给定半径 1800 米的圆形目标区域，机器狗从圆心出发，携带测向机、光学探测仪和激光清除设备，在不知道实际源数和位置的条件下自主决策 [1]。任务目标不仅是找到若干个源，还要给出足以结束搜索的证据，并在全部清除的前提下缩短总任务时间。

20 个频道中每个频道至多对应一个源，实际源总数为 10 至 16 个。全向源在全部方向发射，定向源仅在未知的闭 180 度角域内发射；有效接收半径分别固定于 1000 至 1500 米之间。不同源互不干扰，因此频道提供了可识别的目标身份，但不能提供“该频道是否有源”的先验答案。

机器狗速度为 5 米/秒，每次测向耗时 5 秒，改变测向频道另耗时 1 秒。光学探测耗时 3 秒，距离源不超过 20 米时再耗时 2 秒清除。移动与检测不能同时进行。这些条件使距离最短、观测次数最少与总时间最短成为不同的目标，须在同一个代价模型中权衡。

1.2 四个问题的输入与输出

表 1 四问任务分解

问题	已知信息	主要输出	验证重点
一	检测点和示向度	交会区域直径算法、同直径圆覆盖结论	退化状态、最远点对及反例
二	全向源的一次有效示向	第二测点候选域及选择策略	保证接收、定位精度与移动量
三	全部源为全向，具体数目未知	搜索、定位、清除及停止策略	全清比例、平均定位清除时间
四	全向与定向混合，类型与方向未知	对不可见性稳健的完整策略	定向不漏检、成本波动与时间

前两问研究单源几何信息如何转化为决策，后两问将其嵌入多频道在线任务。问题一回答可行域如何表示；问题二回答下一次观测怎样选择；问题三建立完整搜索与调度；问题四在不可见性增加后修改证据规则和执行方式。四问共享物理参数，但各问的可行域定义不能混用。

1.3 研究对象与评价指标

本文以题设协议和本文方法程序为实现对象 [3]。主要指标为全清比例和每个实际清除源的平均任务时间。虚拟任务时间反映移动及设备操作，现实运行时间反映程序计算和通信；两者分别记录。后文的合成模拟、用户提供的演练汇总和正式测试表互相独立，避免用不同来源的数据拼成一次完整成绩。

2 问题分析与整体思路

2.1 三个相互耦合的困难

第一，误差无法通过同点重复采样消除。题设说明同一地点的电磁误差固定。将反复读数当作独立样本并求平均，会人为缩小置信区间。本文保留每次测角对应的整个角区间，通过不同位置的约束相交压缩候选源域。

第二，无信号不是无源。对全向源，无信号还可能表示超出半径；对定向源，还可能落在发射背面。一个频道只有在全部合法源状态都与实际观测矛盾时，才能从搜索任务中删除。尤其不能因为已经清除 10 个源，就停止检查其他频道。

第三，定位精度与任务费用不等价。机器狗可能已经处于能覆盖整个可行域的清除点，却继续为缩小几何直径而移动测量；也可能逐源采用看似最优的定位方案，却让源间往返距离增加。因此，单源动作与多源访问顺序需要同时考虑预计服务出口。

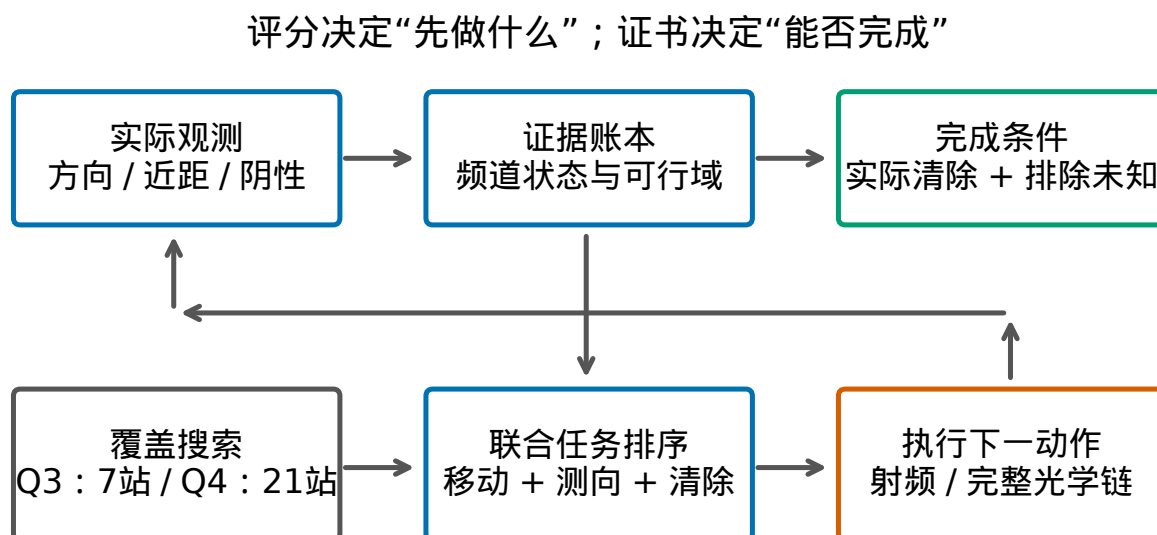


图 1 统一决策流程。实际响应进入证据账本；几何证书和数量上界约束完成判断；费用评分用于安排可执行动作。箭头表示数据或执行结果的传递。

2.2 模型之间的连接

整体流程分为四步。首先，将接收、方向和清除结果转化为集合约束，维护每个频道的状态。其次，用搜索布局或历史证书保证未知源仍有被发现的途径。再次，在剩余搜索与已知源之间选择下一项任务，并在任务内部比较射频测向和光学清除。最后，对动作的预算、实际返回及完成条件做检查，再进入下一轮。

本方法的核心结果是：启发式评分可以影响访问次序，但不直接消除一个可能存在的源。即使概率估计不准，几何责任区域、实际清除响应和完整备用策略仍决定能否宣告完成。因而可以优化平均费用，同时满足覆盖和实际清除的完成条件。

3 题设条件、建模约定与符号

3.1 采用的条件与补充约定

1. 一次案例中，源的位置、频道、有效半径与定向方向固定；不同频道的证据分别维护。该约定对应题目物理环境，不假设源在运行中迁移。
2. 采用平面欧氏距离，机器狗可沿直线移动至目标圆域外。圆域限制源的位置，并不限制机器狗位置，故问题四外环站可位于 1800 米圆外 [3]。
3. 测角误差按任意有界扰动处理，不假定不同检测位置的误差独立、正态或均匀。理论原始误差为 1° ，程序考虑两位小数舍入后采用 $\varepsilon = 1.005^\circ$ 。
4. 证明与执行分开：几何计算采用有理数和向外包络，保证清除判据采用 19.8 米半径；实际接口的成功阈值仍是 20 米。
5. 接收概率评分中使用均匀半径和朝向、全向与定向各半的工作假设。这是行动排序的近似模型，题设未给出这些分布，不将其作为排除证据。

3.2 主要符号

表 2 跨问题共用的符号

符号	含义	单位
$\mathcal{D}, x_c$	源分布圆域、频道 c 的源位置	m
N, R_c, n_c	源总数、接收半径、定向单位法向量	个、m、无量纲
p, h	机器狗当前位置与当前测向频道	m、频道编号
$u(\theta), v(\theta)$	方位单位向量及其垂直单位向量	无量纲
$P_c, \hat{P}_c$	精确相容位置集合及其保守外包	m 坐标集合
U, F, C, A	未知、已发现未清除、已清除、已排除频道集	集合
Q_3, Q_4	全向及混合场景的预设搜索站集	m 坐标集合
M, S, K_s, K_f	测向、切频、光学成功及失败次数	次
L, T_v, T_r	总行进距离、虚拟时间、现实运行时间	m、s、s
E, Φ	剩余试探额度与任务下降秩	非负整数
$\bar{T}, \eta$	平均定位清除时间与实际全清比例	s/源、无量纲

所有角度在文字中用度表示，在三角函数内部换算为弧度。局部二维叉积记作 $[a, b] = a_x b_y - a_y b_x$ 。不同问题的辅助变量在引入处定义，避免把可行域直径、接收半径和清除半径都记成同一个字母。

4 统一观测模型与时间目标

4.1 可接收、方向与近距观测

令 $\mathcal{D} = \{x \in \mathbb{R}^2 : \|x\| \leq 1800\}$ 。全向源在位置 q 可被接收, 当且仅当 $\|q - x_c\| \leq R_c$ 。定向源还须满足

$$n_c^\top(q - x_c) \geq 0, \quad \|n_c\| = 1. \quad (1)$$

这里 n_c 是从源指向有效发射半平面的法向量, 不能与检测点指向源的测向向量混淆。

若可接收且距离大于 5 米, 响应为方向值 θ , 真实源落在以 q 为顶点、方向区间 $[\theta - \varepsilon, \theta + \varepsilon]$ 为边界的前向楔形内。若可接收且距离不超过 5 米, 返回近距响应。若不可接收, 返回无信号。近距响应直接给出 5 米圆盘, 不应人为补出一个方向值。

4.2 动作费用与频道状态

设前后动作位置为 p, q , 当前测向频道为 h 。一次测向与一次光学动作的费用分别为

$$t_m(p, q, c, h) = \frac{\|q - p\|}{5} + 5 + \mathbf{1}_{c \neq h}, \quad (2)$$

$$t_o(p, q, c) = \frac{\|q - p\|}{5} + 3 + 2\mathbf{1}_{\text{清除成功}}. \quad (3)$$

光学动作指定目标频道, 但不改变测向机的当前频道, 也不产生切频费用 [3]。进入和主动退出不推进虚拟时钟。于是整场费用严格分解为

$$T_v = \frac{L}{5} + 5M + S + 5K_s + 3K_f. \quad (4)$$

该公式用于逐条响应回放, 是结果表和费用图的共同计算依据。

4.3 优化与完成约束

令策略 π 根据截至当前的真实观测历史 H_t 选择下一动作。理想目标是对所有合法源状态保证 $K_s = N$, 并使 T_v 尽量小。本文实际采用有界计算预算的在线近似决策, 而不求整个连续信念空间的精确最优策略。

$$\eta = \frac{K_s}{N}, \quad \bar{T} = \frac{T_v}{K_s} \quad (K_s > 0). \quad (5)$$

由于接口不直接返回 N , η 只能在演练结束获得真值后核验。在线停止条件由频道证据定义, 不能以预测的 $\eta = 1$ 替代。

现实运行时间还包括求解和 HTTP 通信。算法始终服从进入时返回的剩余窗口; 本文的几何完备性结论以有效响应、正确数值包络和足够执行时间为条件, 不据虚拟时间较小推断现实窗口必然满足。

5 问题一：交会定位区域与直径

5.1 前向楔形的半平面表达

令第 i 个检测点为 p_i ，示向度为 θ_i ， $u(\theta) = (\cos \theta, \sin \theta)$ 。对应的前向楔形为

$$W_i = \{x : [u(\theta_i - \varepsilon), x - p_i] \geq 0, [u(\theta_i + \varepsilon), x - p_i] \leq 0\}. \quad (6)$$

因为 $2\varepsilon < 180^\circ$ ，上述两式确定的是朝向测量方向的凸楔形。若只保留“靠近示向直线”的条件，会把检测点背后的对称区域错误地纳入定位区域。

将每个楔形写成两条线性不等式，得到

$$P = \bigcap_{i=1}^n W_i = \{x : a_j^\top x \leq b_j, j = 1, \dots, 2n\}. \quad (7)$$

问题一计算的对象是这个纯交会区域。用于后续行动时可以再与源圆域、已接收半径上限等先验相交，但不能在回答问题一时悄悄加入人工边界盒，使原本无界的交集变成一个有限多边形。

5.2 空、无界与有界状态的分类

枚举所有非平行边界线对。若 a_j, a_k 线性独立，其交点通过二元线性方程求出；只有满足全部 $2n$ 条不等式的点才是可行候选。存在至少一个尖楔形时，非空交集不含整条直线，因此存在极点。精确算术下，找不到可行候选可以判空；浮点实现对近平行数值情况保留未决标记。

对非空交集，再考察衰退锥

$$\mathcal{R} = \{d : a_j^\top d \leq 0, j = 1, \dots, 2n\}. \quad (8)$$

若存在非零 $d \in \mathcal{R}$ ，则对可行点 x_0 有 $x_0 + td \in P$ ， $t \geq 0$ ，故直径无穷。二维尖锥的极射线与某约束边界平行，可枚举约束法向量的两个垂直方向并检验。没有观测时直接返回全平面无界状态。

5.3 实现中的数值判据

纯交会模块采用浮点边界枚举，对可行性使用相对容差；近平行交点、无穷值及空集按状态区分返回。后两问的证书模块另外使用有理数裁剪。本文不把前者的浮点直径输出解释为严格区间证明；这一区别也决定了问题一结果的保留小数位和后续行动的安全余量。

5.4 直径计算与复杂度

命题 5.1. 若 P 是非空有界凸多边形, 顶点为 $z_1, \dots, z_m$, 则

$$\text{diam}(P) = \max_{i,j} \|z_i - z_j\|. \quad (9)$$

证明. 任意 $x \in P$ 可写成顶点的凸组合 $x = \sum_i \lambda_i z_i$. 对固定 y , 三角不等式给出 $\|x - y\| \leq \sum_i \lambda_i \|z_i - y\| \leq \max_i \|z_i - y\|$. 再对 y 作凸组合展开, 即可把最大值同时推到两个顶点上. $\square$

对候选点求逆时针凸包后, 采用旋转卡壳枚举对踵点对 [4]. 边界交点枚举有 $O(n^2)$ 对, 每点检验 $O(n)$ 个条件, 因此整体为 $O(n^3)$; 顶点凸包为 $O(m \log m)$, 卡壳扫描为 $O(m)$. 在本题小规模观测下, 直接枚举易于检查, 也能保留清楚的退化状态。

表 3 交会区域直径算法

步骤	运算与输出
1	为每次示向生成前向楔形的两条半平面约束。
2	枚举非平行边界交点, 检验全部约束, 保留可行点。
3	若无可行点, 返回空或数值未决; 若存在非零衰退方向, 返回无界。
4	对剩余点求凸包; 单点直径为 0, 线段直径为端点距离。
5	对多边形执行旋转卡壳, 返回最大距离和对应最远点对。
6	用最远点对中点及所有顶点检验同直径圆的覆盖性。

5.5 同直径圆的充要检验

设最远点对为 a, b , $D = \|a - b\|$. 任何半径 $D/2$ 且覆盖 a, b 的圆, 其圆心必须是 $c = (a + b)/2$. 因此存在同直径覆盖圆的充要条件是

$$\max_j \|z_j - c\| \leq D/2. \quad (10)$$

只检验这两个最远点不够; 所有其他顶点也必须满足条件。若条件不成立, 可另求最小包围圆或使用更保守的半径, 但不能把 $D/2$ 直接作为保证覆盖半径。该结论为后续“整个可行域均在 20 米内”的清除判据提供了必要修正。

5.6 可实现的反例与算例

考虑边长为 d 的正三角形。其区域直径为 d ，最小包围圆半径却为 $d/\sqrt{3}$ ，大于 $d/2$ 。故问题一的第二问答案为**不一定能覆盖**，且失败并非浮点误差造成。

这一反例可以由符合测向结构的楔形实际产生。沿单位正三角形各条逆时针边向后延长 100 米设置检测点，将示向度设为该边方向再加 1° ，误差半宽取 1° 。每个楔形的一条边界与三角形边重合，另一条边界不切入三角形；三个前向楔形之交恰为该三角形。这样，反例既满足凸几何性质，也对应题目允许的观测形式。

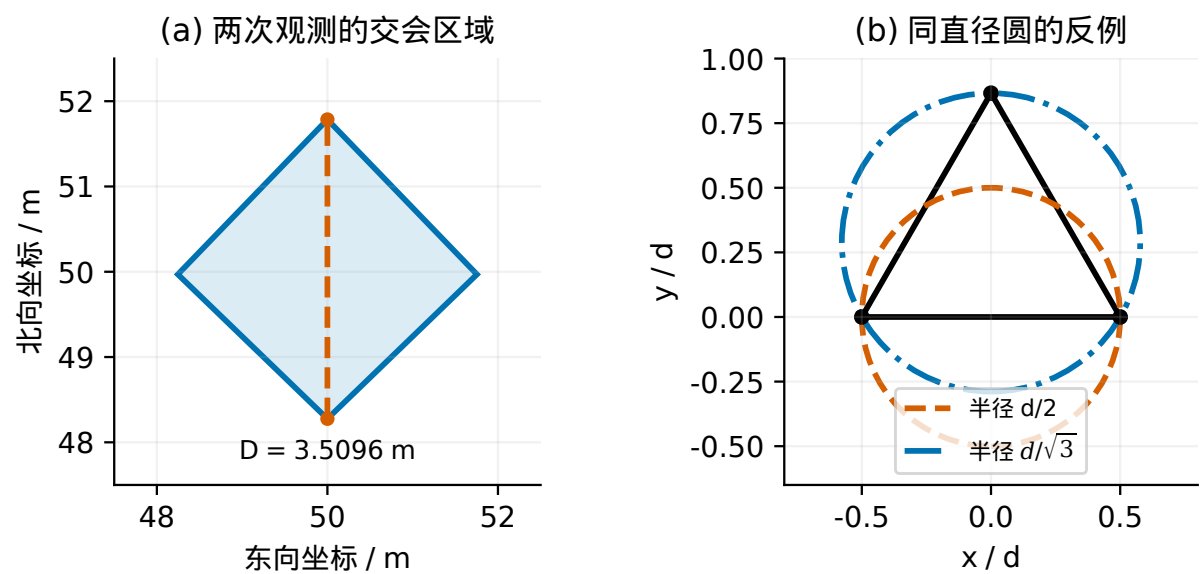


图 2 问题一的数值交会与解析反例。左图取检测点 $(0,0)$ 、 $(100,0)$ 及示向度 45° 、 135° ，误差半宽 1.005° ；虚线连接最远点。右图以边长 d 归一化，虚线半径为 $d/2$ ，点划线为最小包围圆。

表 4 问题一的确定性检验结果

输入情形	输出或理论结果	核验内容
单个正常示向	无界楔形	不引入人工有限边界
前向条件相互矛盾	空或数值未决	不输出虚构有限直径
图中两次示向	$D = 3.5095516$ 米	顶点与最远点对一致
单位正三角形	$D = 1, r_{\min} = 1/\sqrt{3}$	同直径圆覆盖判据不成立

在实际多次测向中，新增有效约束使精确交集满足 $P_{k+1} \subseteq P_k$ ，故直径不会增大。这一集合嵌套性比假设平均误差按样本量缩小更适合本题。若计算结果反而扩张，首先应检查角度环绕、前向方向、单位换算与数值容差，而不能把扩张当作真实源运动。

6 问题二：第二检测点的候选域

6.1 首次接收给出的半径耦合

设首次正常测向位置为 p_1 、示向度为 θ_1 。在局部正交基 $u = u(\theta_1)$ 、 $v = v(\theta_1)$ 下，真实源可写为

$$x = p_1 + r(\cos \delta u + \sin \delta v), \quad 5 < r \leq 1500, \quad |\delta| \leq \varepsilon. \quad (11)$$

因为首次已经接收到信号，未知半径并非独立自由参数，而满足 $R \geq \max\{1000, r\}$ 。设第二点为 $q = p_1 + au + bv$ ，选择前向半平面 $a \geq 0$ 。把 r 放宽到 $[0, 1500]$ 后，保证接收要求

$$\|(a, b) - r(\cos \delta, \sin \delta)\| \leq \max\{1000, r\}, \quad \forall r, \delta. \quad (12)$$

6.2 消去连续距离变量

当 $0 \leq r \leq 1000$ 时，距离平方关于 r 为凸函数，最大值出现在 $r = 0$ 或 $r = 1000$ 。当 $r \geq 1000$ 时，平方后抵消 r^2 ，式(12)等价于

$$a^2 + b^2 \leq 2r(a \cos \delta + b \sin \delta). \quad (13)$$

若 $r = 1000$ 时成立，右侧系数必非负，增大 r 不会使约束更紧。对 $a \geq 0$ 及小角区间，最不利角度是两个区间端点。于是只需检查三个圆盘：

$$\begin{aligned} \mathcal{C} = & B((0, 0), 1000) \cap \{a \geq 0\} \\ & \cap B((1000 \cos \varepsilon, 1000 \sin \varepsilon), 1000) \\ & \cap B((1000 \cos \varepsilon, -1000 \sin \varepsilon), 1000). \end{aligned} \quad (14)$$

该集合是放宽先验和前向条件下的完整保证接收域；对真实 $r > 5$ 及源圆域约束，它是保守的充分候选域。若首次响应为近距，不需要使用此构造，可直接在原位置清除。

6.3 相对于独立最坏半径处理的改进

若忽略首次接收事实，要求全部 1500 米端点都落在第二点的 1000 米接收圆内，会排除许多合法候选。例如 $(a, b) = (500, 840)$ 满足式(14)，却不满足上述独立处理。其改进来自保留观测造成的变量耦合，而不是放松真实接收约束。

6.4 定位精度的解析上界

候选域保证能够接收，但不自动保证交会角度良好。令 $\rho = \sqrt{a^2 + b^2}$ 、 $\alpha = \text{atan2}(b, a)$ ，先考虑 $b > 0$ 的一侧。真实源到第二点的距离上界为

$$d_2(\rho, \alpha) = \max_{r \in \{5, 1500\}} \sqrt{r^2 + \rho^2 - 2r\rho \cos(\alpha + \varepsilon)}. \quad (15)$$

第二点相对于首次方向的最小横向高度为 $h = b \cos \varepsilon - a \sin \varepsilon$ 。若 $h > 0$ ，两条名义测向方向的最小交角可保守估计为

$$\gamma = \arcsin(\min\{1, h/d_2\}) - 2\varepsilon. \quad (16)$$

当 $\gamma \leq 0$ 时，无法由此得到有限的条带交直径上界。令两条包络条带的半宽为 $w_1 = 1500 \sin \varepsilon$ 、 $w_2 = d_2 \sin \varepsilon$ ，则最宽对角线给出

$$D_{\text{ub}} = \frac{2\sqrt{w_1^2 + w_2^2 + 2w_1w_2 \cos \gamma}}{\sin \gamma}. \quad (17)$$

这是对物理相容区域的外包估计。缩小该上界有助于获得稳健交会，但它与真实区域直径的最小化不是同一问题。

6.5 两种第二点选择方式

给定移动预算 ρ 时，在可行角区间

$$\varepsilon < \alpha < \arccos(\rho/2000) - \varepsilon, \quad 5 < \rho < 1000 \quad (18)$$

内最小化式(17)。给定期望直径阈值 D_0 时，则求

$$\min_{\rho, \alpha} \rho \quad \text{s.t.} \quad D_{\text{ub}}(\rho, \alpha) \leq D_0, (a, b) \in \mathcal{C}. \quad (19)$$

实现按区间维护目标下界和当前可行上界：不能改进当前解的区间剪除，其余区间二分。角度方向利用 d_2 与 h 的单调界；二维移动优化对距离和角度同时建立保守区间界。该思想与有界优化中的可行上界及松弛下界相一致 [5]。

获得 (a, b) 后，另一个对称解为 $(a, -b)$ 。若给定当前机器狗位置，选移动费用较小的一侧；否则使用固定侧作为可复现的规则。比较双方时不改变接收和精度约束。

6.6 计算结果及定位代价的解释

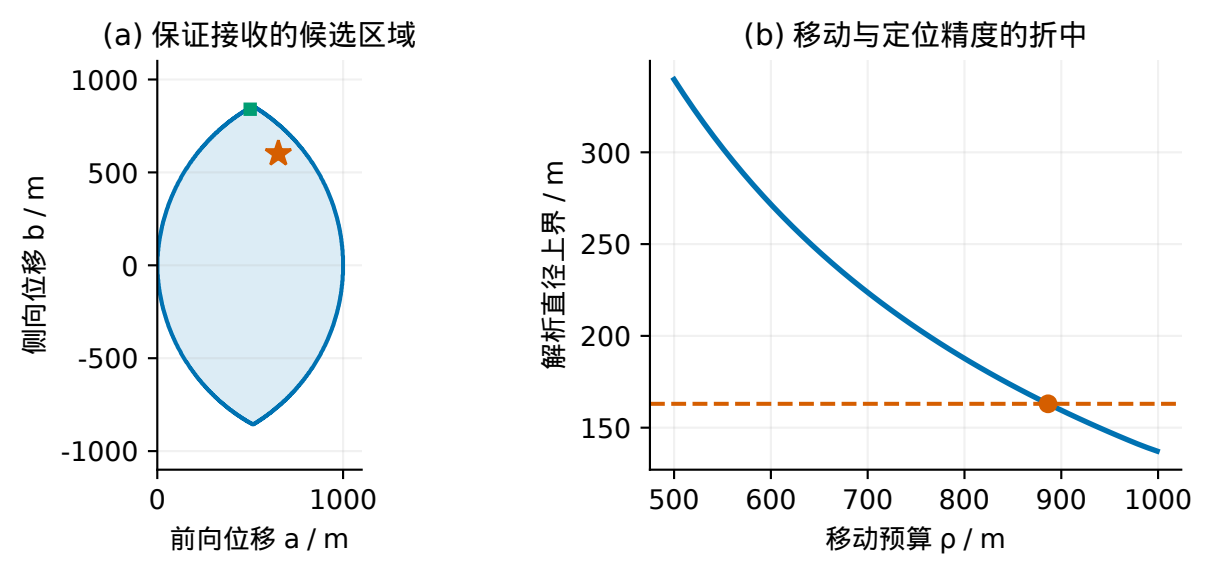


图 3 第二测点的候选域和移动精度折中。左图浅色区域为三个圆盘与前向半平面的交，星号为 163 米解析目标下的可行解，方点为 (500, 840)；右图曲线由当前解析优化程序逐点计算，虚线表示 163 米目标，不是演练实测曲线。

表 5 问题二的解析优化结果

量	计算值	单位
目标直径上界 D_0	163.0000	m
可行移动距离 ρ_U	886.3434	m
移动距离计算下界 ρ_L	886.2614	m
上下界间隙 $\rho_U - \rho_L$	0.08194	m
前向、侧向位移 (a, b)	(651.5213, 600.9365)	m
实际解析上界 D_{ub}	162.9998	m
保证接收的距离余量	113.6566	m

本问的主要数值结果是：在解析直径上界不超过 163 米这一判据下，最小移动距离已被限制在一个宽约 0.082 米的区间内。这只证明该解析设计问题的计算间隙很小，不证明连续源状态下所有自适应定位策略都已达到理论最优。

固定 900 米移动预算的另一种方案给出解析直径上界 159.4987 米。与 886.3434 米方案相比，它进一步压缩定位区，却增加了首段移动。后两问还包含多源移动和 20 米光学清除阈值，因此不能机械地把单源第二点固定为某一“精度最优点”。若当前观测已经足以产生保证清除位置，就应直接比较清除费用。

完整的实际定位次数、最终平均清除用时以及该设计相对固定第二点的演练增益尚需同输入试验，分别预留为_____、_____和_____。这些未测值不影响候选域的几何推导，但会影响第二点策略在整场任务中的经济性评价。

7 问题三：全向源的完整搜索与联合清除

7.1 七站布局的构造

取原点以及半径 1125 米圆周上等角分布的六个点：

$$Q_3 = \{(0, 0)\} \cup \{1125(\cos(k\pi/3), \sin(k\pi/3)) : k = 0, \dots, 5\}. \quad (20)$$

选择 1000 米最小有效半径作为保证依据，而不使用 1500 米最大半径。离原点不超过 1000 米的源由中心站覆盖；其余源选择方位差不超过 30° 的外环站。

命题 7.1. $\mathcal{D} \subseteq \bigcup_{q \in Q_3} B(q, 1000)$ ，故对任意合法全向源，只要在七站逐一检查其频道，必有一次可接收观测。

证明. 设源的极径为 $r \in [1000, 1800]$ ，最近环站与其方位差为 $\delta \leq \pi/6$ 。距离平方不超过 $r^2 + 1125^2 - 2250r \cos(\pi/6)$ 。该式关于 r 为凸函数，最大值在两个端点取得；较大者对应 $r = 1800$ ，距离约 999.1107 米，小于 1000 米。中心站覆盖其余区域。 $\square$

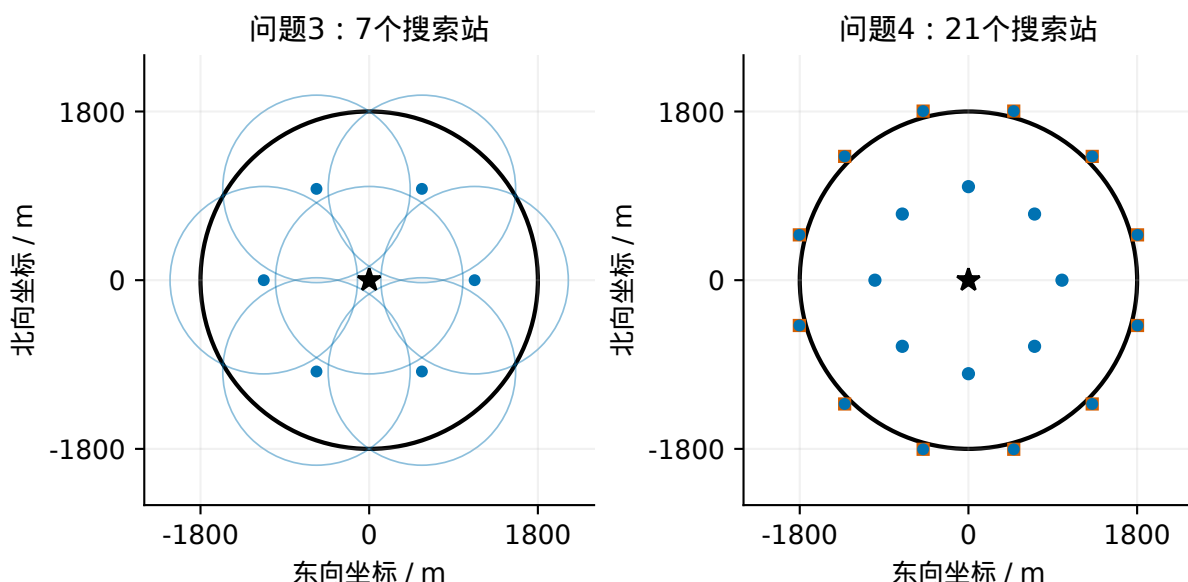


图 4 搜索站布局。左图的浅色圆为 1000 米接收圆，覆盖 1800 米源域；右图为问题四的 21 站，其中外环半径 1868 米。点集只提供初始覆盖结构，实际访问顺序随任务状态调整。

7.2 覆盖与访问顺序的区别

上述命题证明“到这些站逐频道搜索足以发现”，不要求必须把七站走完。已经发现 16 个不同源时，数量上界允许排除其余频道；历史实际观测或清除点若能替代某个剩余搜索站，也可以缩短路线。

程序对删站或替换站后的历史证据点与未来站点之并集重新做连续覆盖核查。通过只表示新的搜索计划具备完整发现能力，不会把尚未走到的未来站点记成实际阴性证据。因此路线可以提前优化，账本中的“无源”结论仍必须等待对应真实观测。

7.3 频道状态与证据账本

每个频道具有四种互斥状态：未知 U 、已发现未清除 F 、已清除 C 、已排除 A 。每次真实观测只更新被请求的频道。几何上接近的两个源也不能交换频道证据，因为不同频道对应不同物理对象。

表 6 频道状态转移规则

当前状态	实际返回或证明	更新方式
U	方向或近距	转为 F ，建立保守可行域和清除责任块
U	无信号	保存该点阴性；覆盖证明成立时转为 A
F	新方向、近距或有效阴性	缩小相容域，更新责任块，不直接标为已清除
F 或 U	光学清除成功	转为 C ，保存实际成功证据
F	光学未命中	删除已被该光学圆完整覆盖的责任块
U	已发现不同源达 16 个	依据题设数量上界转为 A

令 $D_t = F_t \cup C_t$ 。合法停止条件为

$$U_t = \emptyset, \quad F_t = \emptyset, \quad 10 \leq |C_t| \leq 16. \quad (21)$$

若 $|D_t| = 16$ ，其余未知频道可依据唯一频道和数量上界排除；若只发现 10 至 15 个源，则仍存在额外合法源，不能仅据已发现数量退出。

7.4 全向阴性如何压缩已知源域

设 p 为实际接收点， q 为同频道无信号点。全向源具有固定半径 R ，所以 $\|x - p\| \leq R < \|x - q\|$ 。平方后得到

$$2(q - p)^T x < \|q\|^2 - \|p\|^2. \quad (22)$$

实现保留非严格外包半平面，避免边界舍入误删真实源；同时可删除被 $B(q, 1000)$ 完整包含的责任块。式(22)使用了已接收和未接收的相对信息，比单纯记录“在此点没有方向”更有价值。

与此相对，问题四的无信号不能直接使用式(22)，因为无信号也可能来自定向背面。两种场景采用不同的阴性更新规则，是保证算法正确性的关键。

7.5 有限责任块与备用完成过程

正常方向观测给出长度不超过 1500 米的细长可行域。程序沿测向轴布置 61 个纵向位置，横向取 $-25, 0, 25$ 米三层，最多生成 183 个责任块。每个块只保留与当前可行多边形的交，且在建立时核查整个块均落在其对应清除点的 20 米圆内。近距观测只需要一个责任块。

未发现频道另外保留覆盖 $[-1800, 1800]^2$ 的 49 个 600 米网格点。备用策略沿固定蛇形顺序检查未知频道，一旦发现源就执行其有限光学责任序列。网格每个 600 米方格的四角距离均不超过 $600\sqrt{2} < 1000$ 米，且源位于四角凸包内，所以该备用点集也适用于定向情况。

7.6 下降秩与有限性

令 G_c 为未知频道尚未排查的备用网格点， B_c 为已知源尚未消除的责任块， E 为剩余可选试探额度。定义

$$\Phi = \sum_{c \in U} |G_c| + \sum_{c \in F} |B_c| + 183(16 - |D_t|) + E. \quad (23)$$

新发现源最多增加 183 个责任块，同时使第三项减少 183；网格排查、责任块清除或实际源清除会减少相应任务量。若动作没有使上述几何任务量严格下降，必须消耗一个不可重置的试探额度。因此，每次被接受的实际行动均使非负整数 Φ 严格下降。

初始 49 个网格点、20 个频道和 $E = 640$ 给出 $\Phi_0 = 4548$ 。这提供有限动作数的粗上界，不能解读为所有场景都需要这么多动作。任务结束时 Φ 也不必为零，因为实际源数可能小于 16，数量潜力项仍有剩余。

7.7 预算检查与保证的适用范围

每次可选动作前，计算当前状态的保守备用完成费用 $U(H_t, p)$ ，把额外移动、操作和潜在新源清除费用计入检查。若预算不足、规划到期或可选试探不足，则切入备用策略。实际执行还预留请求延迟和退出窗口。

由相容域包含真实源、责任块覆盖及下降秩，可以建立条件性的完整发现和清除结论：在题设物理规则、有效通信及足够执行窗口下，备用过程不会因有限次失败而丢失源。通信中断或现实时间耗尽不属于这个几何命题的保证范围，需在结果中单独报告。

7.8 搜索与已知源的统一服务任务

把剩余搜索站和已知未清除源视为一组任务 $\mathcal{J}$ 。每个任务 j 可具有若干服务模式 m ，分别描述入口 e_{jm} 、预计出口 x_{jm} 以及服务费用 s_{jm} 。搜索任务的入口和出口均为站点，服务费用保守取 6 乘以需要检查的频道数；已知源任务则提供直接清除或不同测向入口方案。

对任务序列 σ 及模式序列 m_i ，用下式评价完整计划：

$$\hat{T}(\sigma, m) = \frac{\|p - e_{\sigma_1 m_1}\|}{5} + s_{\sigma_1 m_1} + \sum_{i=2}^{|\mathcal{J}|} \left[\frac{\|x_{\sigma_{i-1} m_{i-1}} - e_{\sigma_i m_i}\|}{5} + s_{\sigma_i m_i} \right]. \quad (24)$$

入口是计划首先到达的位置，出口则是结束该项服务后的估计位置。源域中心仅用于估计出口，不作为真实源坐标输入证据账本。因此式(24)是排序代理目标，不是整场费用的严格上下界。

7.9 多个初始顺序与模式动态规划

程序比较五种顺序：上一轮仍有效的顺序、贪心顺序、先处理源、先处理搜索站，以及指派松弛经子环连接获得的顺序。指派问题用原始对偶势方法求解，其基础是匈牙利方法 [6]；取消子环约束得到的指派值不能直接充当真实定位任务的最优值。

对固定任务顺序，定义 $f_i(m)$ 为到第 i 项任务并使用模式 m 的最小累计代理费用，则

$$f_i(m) = s_{\sigma_i m} + \min_k \left\{ f_{i-1}(k) + \frac{\|x_{\sigma_{i-1} k} - e_{\sigma_i m}\|}{5} \right\}. \quad (25)$$

这一递推只优化给定顺序的模式组合。随后进行至多两轮单任务移位，将一个任务插入另一位置，并对较有希望的候选重新评价。计算量受到固定轮数和候选数约束，适合实时更新。

7.10 为何一次只承诺当前任务

新观测会改变源域、接收可能性和剩余未知频道，过早固定整场路线容易过时。算法选定当前任务后执行有效前缀；对源任务通常持续处理到清除，再重排剩余任务，同时将下一任务入口作为局部清除出口的参考。这样利用全局顺序信息，又保留新证据到来后的调整能力。

7.11 源内射频测点与连续清除位置

设当前可见锚点为 o ，示向单位向量为 u ，垂向单位向量为 v ，锚点至源距离上界为 L_c 。只有一次方向观测时，候选点取

$$q_{\pm} = o + [5 + (L_c - 5)\lambda]u \pm (L_c/50)v, \quad \lambda \in \{0.38, 0.50, 0.62\}. \quad (26)$$

已有至少两次阳性时，先将当前多边形投影到锚点轴，得到区间 $[l, h]$ ，再改取轴向中点 $(l + h)/2$ 和侧距 $\max\{5, (h - l)/20\}$ 。这样不会重新把历史已经排除的近距离范围纳入下一步设计。

对候选测点，前瞻枚举方向区间、近距及阴性分支，以分支后的完整备用清除费用评价。未能在规划预算内完成的候选不继续搜索。前瞻估计只帮助选动作；实际账本只能根据真实响应更新。

若整个当前源域都能从某位置清除，则可直接求

$$q^* = \arg \min_q \|q - p\| \quad \text{s.t.} \quad \|q - z\| \leq r_s, \quad \forall z \in \text{vert}(\widehat{P}_c), \quad r_s = 19.8. \quad (27)$$

约束为圆盘交，是凸集 [5]。最近点可能是当前位置、一个活动圆上的投影，或两个活动圆的交点，故可枚举检查；若交为空，继续定位或构造多点光学序列。程序对候选点再次用整个有理多边形验证，不只检查代表点。

7.12 顺路观测和清除出口优化

已到达一个搜索点或清除点后，若测量其他已发现频道能产生足够视差，可在原地补测以节省再次绕行。评价利用当前沿轴宽度 w 、观测距离 d 及视差指标 κ ，估计补测后宽度

$$w' \approx \min\{w, 2\varepsilon_{\text{rad}}d / \max(0.01, \kappa)\}. \quad (28)$$

此式是费用启发量，不构成几何收缩证书。若选定下一目标站 t ，清除点还可在保证可行域内尝试减小 $\|p - q\| + \|q - t\|$ ，避免“到最近清除点后又绕回”造成额外移动。

同站同时发现多个源时，程序还比较 350 米环上的共享第二观察点。问题三按代理收益选较好方向；问题四另加接收概率和净收益门槛，具体见后文。该差异提示：共享动作在问题三中的实际增益仍需消融验证，不能由局部直觉直接保证。

7.13 问题三的离线结果

表7列出本文方法的 15 场固定离线输入，合计 192 个源，均获得实际清除成功响应并正常退出。常规、边界、密集与备用场景分别列出；边界、密集和备用场景的分布及执行方式不同，不把它们混合为一个随机总体的性能估计。

表 7 问题 3 的 15 场离线固定输入结果

案例	源数	总时间/s	平均时间/(s/源)	M	K_f
S1	16	2940.55	183.78	90	0
S2	10	3072.89	307.29	130	0
S3	10	2785.99	278.60	113	0
S4	12	3084.86	257.07	111	0
S5	14	3243.22	231.66	115	0
S6	16	3318.80	207.42	115	3
S7	10	2948.40	294.84	127	2
S8	12	3160.66	263.39	111	1
S9	14	2921.20	208.66	114	4
S10	16	3124.71	195.29	96	1
B10	10	4402.57	440.26	144	2
B16	16	4587.68	286.73	125	5
C10	10	2085.60	208.56	91	0
C16	16	437.67	27.35	33	0
R10	10	15348.44	1534.84	660	249

表中 S1 至 S10 为常规输入，B 为边界输入，C 为密集输入，R 为备用过程；B、C、R 后的数字为源个数。各案例的频道、位置、半径、方向和噪声设置见支撑材料中的合成案例数据。 M 表示测向次数， K_f 表示光学失败次数，均由协议记录统计。

十场 S 类常规输入的平均总时间为 3060.13 秒。其中移动时间平均占比约 76.01%，说明单纯减少一次 5 秒测向通常不足以解释主要提速；任务次序、共享位置和清除出口对总费用更敏感。16 源密集案例耗时 437.67 秒，比其他案例明显小，是因为源集中且达到数量上界后可以结束未知频道搜索，不表示源越多一定越容易。

强制备用案例耗时 15348.44 秒，显著高于主动调度。其作用是验证在前瞻和主动优化停止后仍能接续完成，不是估计正常调度的典型平均值。真实演练、正式测试以及现实运行窗口的结果另列，不用本表代替官方测试成绩。

8 问题四：未知定向下的完整策略

8.1 覆盖必须同时排除位置和方向的不确定性

问题三的七个 1000 米圆盘覆盖源域，只说明每个位置附近有测点。对定向源，唯一附近测点可能处于辐射背面，故圆盘覆盖不能直接推出一定发现。问题四需要对每个源位置考虑所有可能的发射半平面。

命题 8.1. 对任意 $x \in \mathcal{D}$ ，若存在 $Q_x \subseteq Q$ 满足

$$\|q - x\| \leq 1000 \quad (q \in Q_x), \quad x \in \text{conv}(Q_x), \quad (29)$$

则任意合法定向方向至少在一个 $q \in Q_x$ 上可被接收。

证明. 由凸包条件可写 $x = \sum_q \lambda_q q$, $\lambda_q \geq 0$, $\sum_q \lambda_q = 1$ 。对任意定向单位法向量 n ，有 $\sum_q \lambda_q n^\top (q - x) = 0$ ，所以不可能所有项都严格小于零。至少一个测点满足 $n^\top (q - x) \geq 0$ ，且其距离不超过最小接收半径 1000 米。 $\square$

8.2 二十一站布局与连续核查

本文方法采用

$$\begin{aligned} Q_4 = \{(0, 0)\} \cup \{998u(k\pi/4) : k = 0, \dots, 7\} \\ \cup \{1868u(k\pi/6 + \pi/12) : k = 0, \dots, 11\}. \end{aligned} \quad (30)$$

外环允许超过源域边界，作用是从边缘外侧观察朝外发射的源。内外环的角度错开，使边缘源也能从多个方向被观测。

连续证书将源域外包方盒递归细分。对每个尚与源圆相交的小盒，选出能用 1000 米圆覆盖其全部顶点的测点，再检查小盒全部顶点是否位于这些测点的凸包内。圆盘和凸包都是凸集，因此顶点判据可推广到整个小盒。不能通过的盒继续分裂；仅采样中心点通过不算证明。

程序将细分规模限制为 20000 节点、14 层深度。预算耗尽时只返回“未证实”，不能把它转化为无源结论。这种保守判定保留了几何完备性，同时允许对实际历史点做删站替换。

8.3 定向阴性的正确使用

定向情况下，一个实际阳性点 o 与阴性点 a 并不能推出 $\|x - o\| < \|x - a\|$ ： a 可能更近但在背面。因此程序不会将单个定向阴性直接转换为问题三的距离半平面。阴性首先进入接收模型和历史记录，只有满足额外几何条件时才收缩位置域。

8.4 两个阴性点的可验证收缩

设 o 曾实际返回方向值（故 $x \neq o$ ）， a, b 曾实际无信号。对候选源位置 x ，若 a, b 均在其 1000 米内，且 $o - x$ 属于 $a - x, b - x$ 生成的非负锥，则 x 不可能是真实源。

证明. 若源为全向， a, b 在 1000 米内必可接收，已与阴性矛盾。若源为定向，两个阴性在距离已满足时意味着 $n^T(a - x) < 0, n^T(b - x) < 0$ 。而非负锥关系推出 $n^T(o - x) < 0$ ，与 o 曾接收矛盾。边界退化时用闭半平面条件分别检验。□

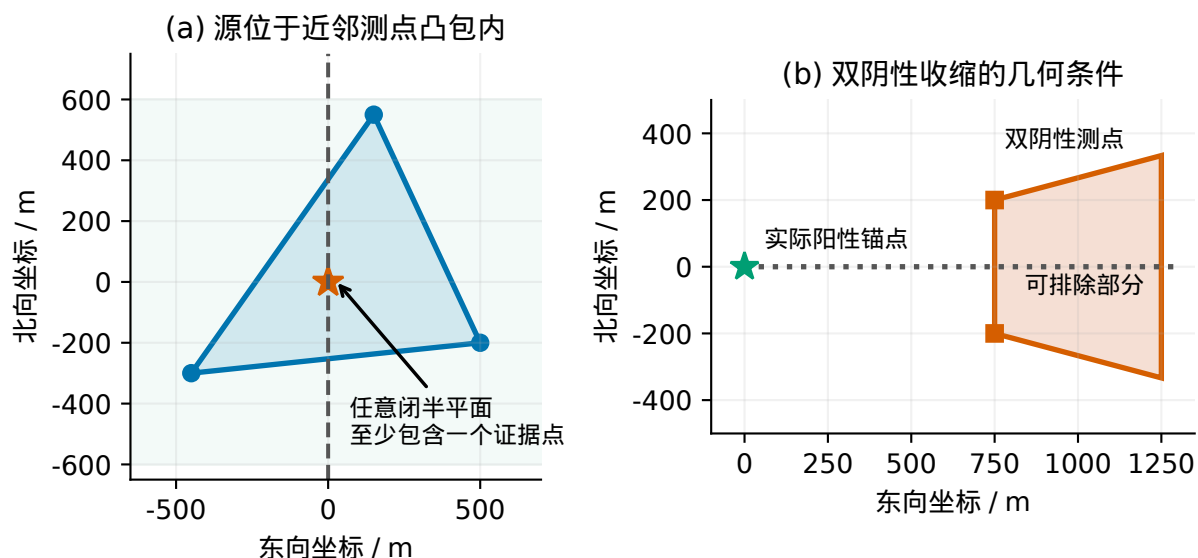


图 5 定向发现与阴性收缩的解析示意。左图源位于近邻测点凸包内，任意发射半平面均有测点可见。右图阴影为双阴性测点连线远离阳性锚点的一侧、且位于相应锥内的示意部分；只有同时通过两个 1000 米圆的包含检验，才可排除。阴影不表示一次实际案例轨迹。

实现用 a, b 连线切出拟排除的多边形，逐顶点核查锥条件及两个 1000 米圆条件。全部通过后才用相应半平面更新源域。位于 o, a, b 三角形内部的候选不能按此理由排除；它可能朝向 o 而背向 a, b ，这是容易混淆的几何边界。

8.5 避免无效重复

同地点电磁环境固定，已在同一源、同一对位置得到双阴性后，无信息变化的重复测量不能改善结论。程序识别这种点对并改用光学责任序列。该处理减少确定性的重复支出；它不允许跳过新的有效搜索点，也不把“重复无信号”累计成更强的统计置信度。

8.6 接收概率只用于动作排序

保证所有源被清除并不要求每一步都取最坏费用。对不会直接删除真实可能性的补测动作，可使用历史观测估计在哪些位置更可能收到信号。本文方法将责任多边形的中心作为位置代表，并以面积赋权；对每个代表位置，再枚举与真实观测一致的半径区间与朝向区间。

对位置代表 x_i 及半径 r ，所有已接收点必须满足距离约束。定向允许朝向集合记为

$$\Theta_i(r) = \bigcap_{p \in H^+} \{\theta : u(\theta)^\top(p - x_i) \geq 0\} \setminus \bigcup_{q \in H^- : \|q - x_i\| \leq r} \{\theta : u(\theta)^\top(q - x_i) \geq 0\}. \quad (31)$$

程序把跨越 0° 的区间拆成普通区间，按实际负观测距离分割半径轴。代表位置与半径段的权重同时考虑面积、段长、剩余定向角度以及全向可能性，归一化后得到候选点 q 的接收概率估计 $\hat{p}_c(q)$ 。

8.7 概率模型的含义与边界

全向/定向各半、半径在区间内均匀、朝向均匀均为工作假设。真实赛题并未给出这些概率分布；位置代表也不能精确积分连续相容域。因此 $\hat{p}_c(q)$ 不是经过校准的客观概率，更不是“未检出源不存在”的概率。

设补测预计把沿轴宽度从 w 压缩至 w' ，代码使用

$$\hat{G}_c(q) = \hat{p}_c(q) \frac{w - w'}{10} - 6 \quad (32)$$

估计净收益，单位为秒。分母 10 把宽度变化粗略换算成剩余移动费用；6 秒包含一次测向及可能切频。它仅为经验代理量，没有被解释为严格的条件期望值。对有足够视差且 $\hat{G}_c(q) > 0$ 的频道，才考虑原地补测。

8.8 多源共享观察点

当同一搜索站至少新发现三个源时，在当前位置 350 米环上每 30 度设置一个候选。比较多个频道的正净收益之和与相对下一目标的额外绕行费用

$$\Delta T(q) = \frac{350 + \|q - t\| - \|p - t\|}{5}. \quad (33)$$

问题四仅在总收益超过绕行代价时执行，并逐频道检查额度和预算。实际返回阴性仍按阴性处理，不用概率预测补造方位；低概率事件也不会跳过完整备用责任。这保证了评分失准主要影响效率，而不改变证据逻辑。

8.9 完整光学链替代继续测向

光学清除与发射方向无关。当可行区域已经足够窄时，为获得另一条示向而绕到可见侧，可能比直接排查整个区域更昂贵。算法因此生成由 2 至 8 个清除点组成的候选链 $Q = (q_1, \dots, q_k)$ ，并要求

$$\hat{P}_c \subseteq \bigcup_{i=1}^k B(q_i, 19.8). \quad (34)$$

候选点沿测向轴及其垂直方向构造。若无法证明式(34)，这条链不进入比较；不能用“所有代表位置都被覆盖”替代连续区域覆盖。

8.10 Voronoi 责任分区的证书

定义离 q_i 最近的责任区域

$$V_i = \hat{P}_c \cap \bigcap_{j \neq i} \left\{ x : (q_j - q_i)^\top x \leq \frac{\|q_j\|^2 - \|q_i\|^2}{2} \right\}. \quad (35)$$

这些区域覆盖整个 $\hat{P}_c$ 。对每个非空 V_i ，若其所有顶点距 q_i 均不超过 19.8 米，则凸性保证整个 V_i 满足同一距离条件。因此逐个有理多边形裁剪和顶点距离比较，可以得到完整光学链的覆盖证书。

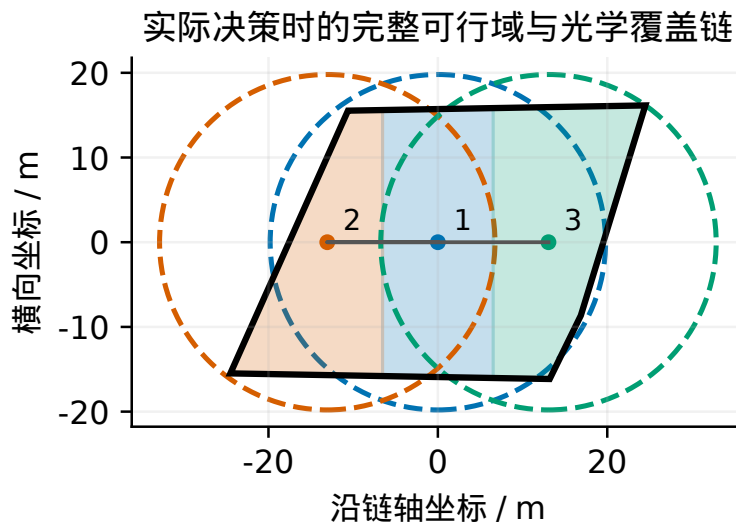


图 6 来自固定离线 Q4 案例的一次实际光学链决策。黑色边界为当时保守源域，虚线圆半径为 19.8 米，填色区为 Voronoi 责任块；数字为候选执行次序。以首点为原点、首末点连线为横轴作刚体变换，不改变覆盖关系。图中不使用源真值。

8.11 为什么执行成功前缀即可

实际源位于至少一个责任区域内，对应清除点必在 20 米以内。算法按序尝试，收到清除成功立即结束该源；此前失败响应用于更新责任区域。覆盖证书约束的是完整链，实际代价却通常只包含成功之前的前缀。这使链的最坏完成保证与平均首次成功费用可以分别评价。

8.12 光学序列的费用比较

给定顺序 Q 及第一次命中第 i 点的估计概率 $\hat{\pi}_i$ ，忽略后续任务时，首次成功的估计费用为

$$\hat{T}_o(Q) = \sum_{i=1}^k \hat{\pi}_i \left[\frac{\|p - q_1\| + \sum_{j=2}^i \|q_{j-1} - q_j\|}{5} + 3(i-1) + 5 \right]. \quad (36)$$

若已确定下一任务入口，还加入从首次成功点到下一入口的预计移动费用。概率由责任块代表和面积权重生成，只用于排序。程序比较轴向顺序、反向顺序和按单位费用命中质量构造的贪心顺序。

程序把较好的光学估计费用与一个射频乐观代理费用比较。该代理假定下一次测向顺利产生定位收益，并估计后续移动与清除费用。如果光学方案更便宜，则在完整覆盖及预算成立时选择它。这不是对真实连续问题的严格支配证明；没有把概率评分包装成“必然优于继续测向”的结论。

8.13 实际执行和保守更新

一次光学失败不能把整个已知源删除。程序只移除被该失败点 20 米圆完整覆盖的旧责任块，其余块保留；随后用剩余块的凸包形成保守外包。该操作可能保留部分已经不可能的位置，代价是多做少量工作，而不会因过度相减丢失真实源。

每个完整光学事件在执行前检查所需试探额度及最坏事件代价。执行过程使用接口返回的响应及累计虚拟时间更新状态。只有清除成功响应能改变“已清除”状态，计划中预测的命中不能直接改变状态。完整链接次序执行，成功响应到达后立即停止后续光学操作。

表 8 问题四的决策层与证据层

环节	用于效率的近似量	必须满足的实际或几何依据
原地补测	接收概率、视差和宽度收益	实际测向返回、额度与窗口
共享第二观察点	多源收益与绕行费用	每频道独立响应，不预判收缩
光学序列排序	代表质量和首次成功估计	完整连续覆盖、实际清除成功
删站与替换	剩余路线和频道检测费用	历史加未来点集覆盖证书
结束运行	不采用期望完成数	式(21)的全清条件

定向场景的时间波动由多个环节共同造成：进入可见侧的绕行、阴性但无法收缩的测量、不同源间的服务出口以及未知频道的最终排除。本文不根据五个不同随机案例的相关性判断其中某一环节的因果贡献，而在后文保留同输入消融表。

8.14 问题四的离线结果与波动解释

当前 15 场 Q4 离线输入合计 198 个源，均实际清除。表9报告总时间、平均每源时间及光学失败次数，含全定向、混合、边界、密集和强制备用输入。所有输入由独立模拟器持有，求解器只访问协议响应。

表 9 问题 4 的 15 场离线固定输入结果

案例	全向/定向	总时间/s	平均时间/(s/源)	K_f
S1	5/11	4551.45	284.47	1
S2	2/8	6229.88	622.99	2
S3	0/10	6365.14	636.51	0
S4	0/12	6534.05	544.50	18
S5	0/14	6399.30	457.09	1
S6	0/16	6680.12	417.51	15
S7	4/6	6381.56	638.16	2
S8	4/8	5939.68	494.97	1
S9	5/9	6359.10	454.22	3
S10	6/10	7176.66	448.54	20
B10	0/10	8060.39	806.04	16
B16	0/16	9206.85	575.43	28
C10	0/10	7148.16	714.82	8
C16	0/16	4290.26	268.14	37
R16	0/16	17850.63	1115.66	893

十场 S 类常规输入平均总时间为 6261.69 秒，移动时间平均占比约 74.01%。与问题三相比，定向约束使“几何上接近”不再保证“可以测向”，进入新视角和排除未知频道的成本明显增加。相比仅根据源个数解释用时，接收结构和最终停止证据更有解释力。

边界 16 源案例耗时 9206.85 秒，说明外向源与边界移动可显著抬高费用；达到 16 源允许停止剩余频道检查，但这一好处未必抵消定位过程的困难。密集 16 源案例仍需 4290.26 秒，也说明位置集中不足以消除方向不可见性。

问题四的一项实际限制是光学链与射频的比较依赖代表点权重和服务出口近似。在真实源位于低权重责任块时，首次成功可能较晚。当前实现通过完整链保证后续仍有清除途径，但其费用受真实源位置和接收方向影响。正式性能评价需同时给出均值、最大值、较差案例和全清比例，而不只挑选最短案例。

9 行动成本与时间分配

9.1 移动与设备操作的统一计量

每个动作的总成本由移动、测向、切频及光学操作组成。移动速度为 5 米每秒，距离 d 对应 $d/5$ 秒；测向 5 秒，切换频道另加 1 秒，光学成功 5 秒，失败 3 秒。程序使用模拟器返回的累计虚拟时间更新运行状态，现实时间另用于限制规划和接口等待。

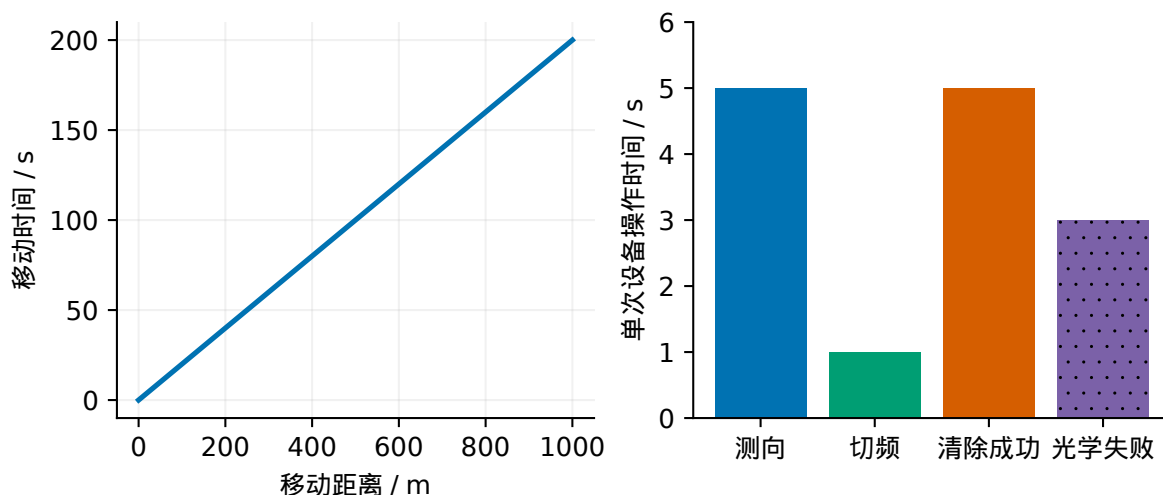


图 7 行动费用的组成。左图给出移动距离与时间的线性关系；右图给出四种设备操作的单次费用。纵轴均为秒，可直接用于同一任务序列的费用累计。

移动 100 米需要 20 秒，已超过一次测向或一次清除的设备费用。因此，减少跨源往返、合并共享观察点、选择合适的服务出口，是控制总费用的主要途径。设备操作仍不可忽略：未知频道较多时，同站逐频道排查会累计测向及切频成本。

9.2 在线计算与备用过程的衔接

射频候选的默认规划窗口为 0.2 秒，可选动作初始额度为 640 次。额度用于限制没有减少剩余任务的试探。当候选计算结束时，立即执行已获得的可用方案；不等待所有可能的后续动作都被枚举。

行动选择还需计入剩余任务的完成成本。程序按尚未排查的网格点、已发现源的剩余清除位置及可能未发现的源数估计备用工作量。若主动动作所需费用与备用过程合计超过剩余预算，则转入备用过程。现实窗口紧张时也采用同一转移，避免把剩余时间耗在规划上。

备用过程先处理已发现源，再沿固定网格逐频道搜索。每次成功清除后释放对应任务；实际没有收到信号时，只更新该频道的记录。初始额度、现实时间和任务计数均随整场运行累计，不因重新规划而重置。

时间分配同时影响效率与完成能力。更长的规划窗口可能改善局部选择，也会占用现实时间；更多试探可能减少移动，也可能产生无效测量。因此参数影响需要在相同输入下单独测量，相关未测结果保留在后面的参数实验表中。

10 计算结果与成本分析

10.1 整体完成情况

30 场合成案例包含常规、边界、密集和备用场景，合计 390 个干扰源，均获得实际清除成功响应并退出。问题三和问题四的完整分场结果分别见表7和表9。这些是合成环境中的计算结果，正式测试结果另列。

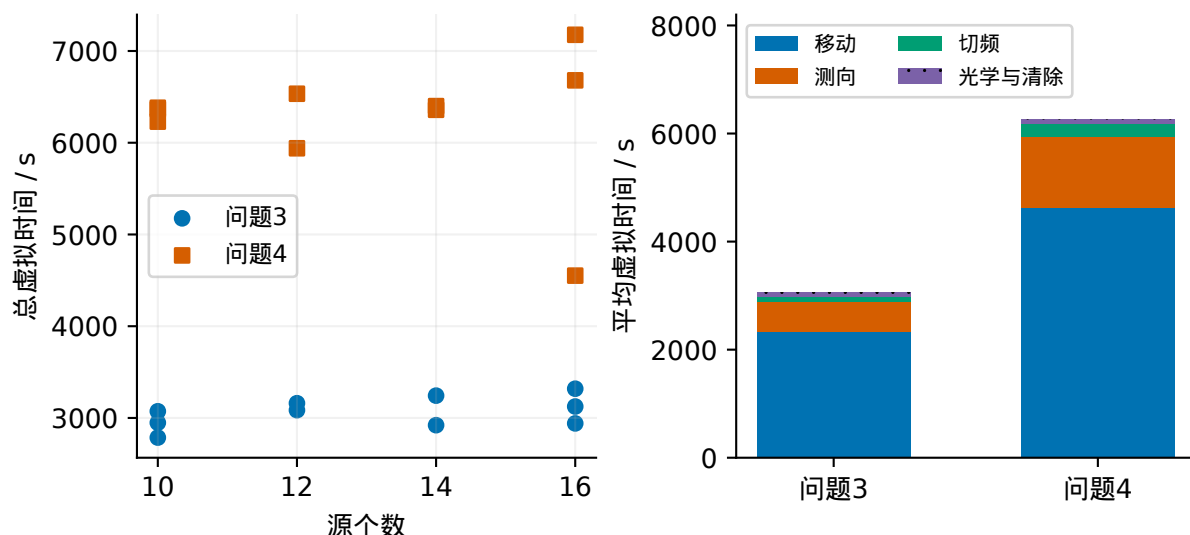


图 8 常规案例的用时与费用构成。左图展示各题 10 场案例的源个数和总时间；右图将平均总时间分解为移动、测向、切频及光学清除费用。同源数的案例具有不同位置和接收条件，散点不表示同一随机量的重复测量。

表 10 常规案例的分题汇总

问题	案例数	平均总时间/s	平均移动时间占比
三	10	3060.13	76.01%
四	10	6261.69	74.01%

两题的移动费用均占总时间的主要部分。问题四中，定位点还必须位于可接收的一侧，几何距离较短并不意味着测向有效。一次阴性结果可能来自接收半径，也可能来自方向限制；无法立即排除大片候选区域时，需要增加视角或改用光学清除。

源个数并不能单独决定总时间。达到 16 个已发现源时，可以利用数量上限排除剩余未知频道；少于 16 个时仍需完成这些频道的排查。位置集中、可接收方向、源间访问距离与最终未知频道排除共同决定时间变化。

10.2 参数敏感性与模块作用

边界与密集输入分别考察接近源域边缘及局部集中分布的情况。两类输入使用固定的极端正负一度误差，同地点重复测量不会产生可平均的新噪声样本。其结果应与常规输入分开分析。

评价参数作用时，需要固定源位置、半径、方向和噪声，仅改变一个参数。下表保留尚未完成的结果，空栏不表示零变化。

表 11 待完成的参数敏感性实验

参数	比较水平	Q3 平均时间/s	Q4 平均时间/s
共享观察半径	250、350、450 m	_____	_____
射频前瞻预算	0.05、0.20、0.50 s	_____	_____
初始可选额度	320、640、960 次	_____	_____
安全清除半径	19.5、19.8、19.9 m	_____	_____

每组都须同时报告全清比例、最差案例时间、现实运行时间和备用触发次数。若缩小安全余量带来时间下降，还必须重新检查连续覆盖与数值边界，不能只比较平均时间。若前瞻预算增加而虚拟时间不变，应检查是否原候选已足够、是否决策主要由备用条件控制，或额外计算没有改变所选动作。

表 12 待完成的同输入消融实验

对照设置	全清比例	相对完整策略 $\Delta T/s$	现实时间/s
关闭原地补测	_____	_____	_____
关闭共享第二测点	_____	_____	_____
关闭任务重排，仅固定站序	_____	_____	_____
关闭 2-8 点光学链，保留完整备用	_____	_____	_____
Q4 接收评分不使用历史阴性	_____	_____	_____

预先约定 $\Delta T = T_{\text{对照}} - T_{\text{完整}}$ ，正值表示该模块在对应输入中有收益。两种方法必须使用相同案例，并保留失败和超时；模块间有相互作用，单项收益不能简单相加。上述分析方法已经确定，具体效应大小留待实际运行后填写。

11 官方演练汇总与正式测试安排

11.1 已经提供的演练记录

参赛队保存了两题各五份演练日志及按运行顺序排列的截图。当前可核对日志的案例编码、题号及时间等公开封装信息，正文为加密数据；尚未从中解析逐步动作与真实源布局。因此表13仅记载截图报告的源个数和整数总时间，不声称已独立解密验证全清比例。

表 13 十次演练截图的成绩汇总

题号/次序	案例编码	报告全向/定向	报告个数	总时间（约）/s
3/1	4GC8-BUAN-JTNR-732X	16/0	16	3233
3/2	ZAJ4-Y9H4-X94F-84RF	11/0	11	2595
3/3	PXX5-MVVF-4U5N-RJ29	15/0	15	3123
3/4	YVXK-H49D-HS89-HBGH	12/0	12	2826
3/5	VD4K-3FT8-EMN3-KHMN	11/0	11	3189
4/1	WZM3-U3MR-YW22-QHNC	0/14	14	7570
4/2	ZG4N-CQJP-AZ7F-6M34	14/2	16	4559
4/3	2AA5-BWCU-M8MH-M9A2	3/8	11	6439
4/4	C2MT-ZV9Z-3G7M-YE4G	2/9	11	7191
4/5	6QJH-B56T-UGBD-NJZ3	9/2	11	6249

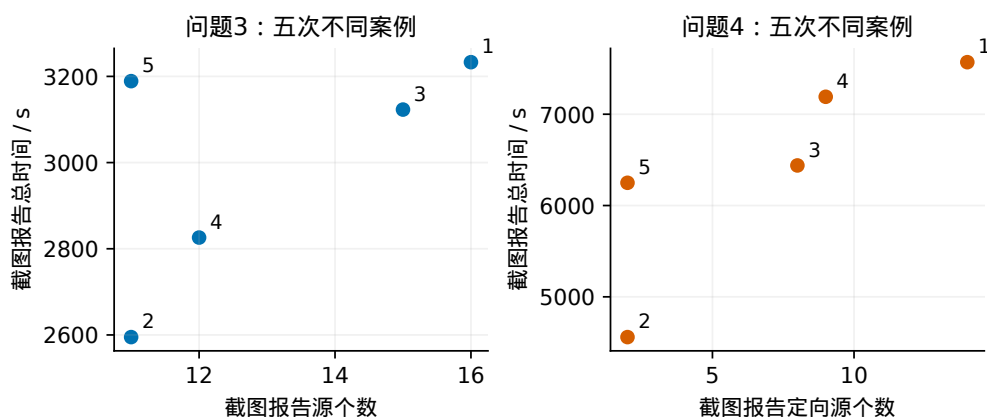


图 9 十次演练截图的描述性比较，点旁数字为运行顺序。总时间为截图取整值；Q4 横轴为报告的定向源数。不同点来自不同案例，未控制位置、半径、方向和噪声，因此未拟合因果趋势线。

问题四时间为 4559 至 7570 秒，跨度 3011 秒，大于问题三的 638 秒。定向较多可能增加绕行与无效测向，但源位置及数量也不同。全向 9 个、定向 2 个的案例仍耗时 6249 秒，说明仅用定向比例无法解释全部波动。需用同次运行的明文动作日志分解移动、测向、失败光学与末尾排除成本，再做同输入消融。

每次演练的实际清除比例和现实运行时间目前分别预留为_____、_____。只有补齐对应原始结果后，才能按题设定义计算平均定位清除时间，不将截图报告源数未经核对地当作清除成功数。

11.2 问题三正式测试结果

题目要求在充分演练后进行三次正式测试，并保留模拟器导出的原始日志 [1, 2]。已取得的正式成绩列于下表，尚未取得的字段保留空栏。测试完成后据实填写相应字段，不用其他演练或离线案例的数值占位。

表 14 问题三正式测试结果（空栏待实测）

测试案例编码	清除源个数	平均定位清除时间/s	程序运行时间/s
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____

11.3 问题四正式测试结果

表 15 问题四正式测试结果（空栏待实测）

测试案例编码	清除源个数	平均定位清除时间/s	程序运行时间/s
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____

11.4 取得数据后的计算与分析

对每次正式测试，先匹配案例编码、题号、策略标识和原始日志，核对实际清除成功数 K_s 及总虚拟时间 T_v ；再按 $\bar{T} = T_v/K_s$ 计算平均定位清除时间，现实运行时间按模拟器给出的记录填写。只有存在实际源总数 N 的可信结果时，才另报 $\eta = K_s/N$ ，不可用程序自身的完成声明替代外部真值。

三次结果的分析同时报告均值、最大值和取值范围。样本数仅为 3，不能据此稳定估计极端尾部概率；若一场明显较慢，应保留该场，检查是否由移动、定向阴性、光学失败或未知频道排除构成主要费用。若发生超时或未清完，则报告实际完成数、结束原因和已消耗时间，不只对成功场计算平均表现。

六份正式日志的原始文件名分别待填入支撑材料清单，不改名、不补造文件。材料整理和论文编译过程均不启动正式模块。本节保留完整的数据取得、指标计算和结果分析框架，待实测数据到位后即可补写具体比较结论。

12 模型评价与结论

12.1 主要优点

证据与费用分层。覆盖证书、实际成功响应及下降秩决定完成性，概率与路径代理决定动作次序。该结构允许近似评分存在误差，并以实际观测和清除结果更新任务状态。

统一考虑定位与移动。第二点设计利用首次接收约束，多源排序考虑服务入口与出口，狭窄源域可直接转为完整光学链。这些处理把几何精度转化为实际任务费用，而不是无限追求更精确的坐标估计。

对定向不可见作显式处理。21 站布局通过连续凸包覆盖核查，单个阴性不被误当成距离证据，双阴性收缩要求真实观测前提。完整备用过程使主动策略在计算预算不足时仍有可接续路径。

12.2 局限与可改进方向

全清结论以题设物理条件、正确数值包络及足够执行窗口为前提；当前入口对通信异常直接停止，没有自动重试。接收评分、源出口和光学首次成功质量均为近似，尚缺独立校准。30 场固定输入检验了实现行为，不能穷尽所有连续位置、半径和朝向；用于开发的输入也不能代替未见场景上的统计保证。

当前下界弱于实际总时间，有限问题最优值只适用于离散动作族。故本文不宣称已逼近连续原问题的全局理论最优。后续可在保持证据账本不变的前提下，先做同输入模块消融，再提高服务费用与接收模型的准确度，并研究包含移动和信息获取的更强下界。

12.3 结论

问题一建立了前向楔形交直径算法，并否定同直径圆必然覆盖的命题。问题二得到保证接收的三圆盘候选域，以及 163 米解析目标下约 886.34 米的可行移动设计。问题三、四分别采用七站与 21 站的连续发现依据，将频道证据、联合任务排序、共享观测和光学清除连接为在线策略。

30 场合成案例中 390 个源全部清除。常规输入中，问题三、四平均总时间分别为 3060.13 秒和 6261.69 秒；不同位置与方向组合产生明显波动。正式测试与尚未完成的敏感性、消融结果均保留待填字段。当前结论由已取得的计算和观测结果支持，新增实测将用于补充效率与可靠性评价。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于算法讨论、代码编写与调试、资料整理、数值核查、图表生成和论文初稿撰写，详细使用情况见支撑材料。本稿的人工修改与逐项核验情况尚需参赛队据实补充，不将自动化测试通过等同于人工审查完成 [7]。

参考文献

- [1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 B 题: 无线电干扰源的快速自动定位与清除 [Z]. 2026.
- [2] 全国大学生数学建模竞赛组委会. 2026 年 B 题附件 1: 模拟器测试说明 [Z]. 2026.
- [3] 全国大学生数学建模竞赛组委会. 2026 年 B 题附件 2: 模拟器通信接口说明及编程指南 [Z]. 2026.
- [4] TOUSSAINT G T. Solving geometric problems with the rotating calipers[C]//IEEE MELECON'83. Athens, 1983: A10.02/1-4. <https://www-cgri1.cs.mcgill.ca/~godfried/research/calipers.html>.
- [5] BOYD S, VANDENBERGHE L. Convex Optimization[M]. Cambridge: Cambridge University Press, 2004. <https://stanford.edu/~boyd/cvxbook/>.
- [6] KUHN H W. The Hungarian method for the assignment problem[J]. Naval Research Logistics Quarterly, 1955, 2(1-2): 83-97. DOI:10.1002/nav.3800020109.
- [7] 全国大学生数学建模竞赛组委会. 全国大学生数学建模竞赛人工智能工具使用规定 (2026 年试行) [EB/OL]. (2026-08-03)[2026-09-13]. https://www.mcm.edu.cn/html_cn/node/fe94648f2836ab6cc81586f4c38512b.html.

附录：支撑材料索引

文件或目录	对应内容
run_robot.py、start_robot.ps1	第三、四题运行入口
src/jammer_solver/	当前算法、精确几何、协议及前两问计算
solve_questions.py	前两问 JSON 输入输出入口
paper/results.json、 paper/cases.json	论文数值汇总与合成案例输入
AI 工具使用详情.pdf	已知 AI 使用环节及人工核验待填记录
正式测试原始.jlog	问题三、四各三份， 待实测生成

正文及参考文献、材料索引共 30 页，之后为完整程序清单。正式原始日志放入 `submission_inputs/formal/` 下的六个对应目录；未测结果及人工核验事项据实补全。所有空栏表示尚未取得对应数据，不表示零值或测试失败。

附录：完整程序清单

以下列出求解、运行、绘图与论文编译所用的完整程序。支撑材料保留相同目录。

run_robot.py

```
1 import argparse
2 from datetime import datetime
3 import json
4 from pathlib import Path as lj
5 import sys
6
7 sys.path.insert(0, str(lj(__file__).resolve().parent / "src"))
8 from jammer_solver.protocol import xjtk
9 from jammer_solver import solver
10
11
12 def main():
13     parser = argparse.ArgumentParser(description="B 题干扰源搜索与清除")
14     parser.add_argument("--problem", type=int, choices=(3, 4), required=True)
15     parser.add_argument(
16         "--case-code", required=True, help=" 模拟器界面显示的案例编码", dest="al"
17     )
18     parser.add_argument(
19         "--robot-id", help=" 运行时输入当前登录的队号，不写入提交源码", dest="dh"
20     )
21     parser.add_argument("--base-url", default="http://127.0.0.1:2026", dest="url")
22     parser.add_argument(
23         "--run-kind", choices=("rehearsal", "formal"), default="rehearsal", dest="lx"
24     )
25     parser.add_argument(
26         "--output", type=lj, default=lj(__file__).resolve().parent / "runs"
27     )
28     parser.add_argument("--connect", action="store_true", help=" 连接已就绪的接口")
29     parser.add_argument(
30         "--planning-seconds",
31         type=float,
32         default=0.2,
33         dest="ghsj",
34     )
35     parser.add_argument("--extra-actions", type=int, default=640, dest="dzys")
36     cs = parser.parse_args()
37     if not cs.connect:
38         print(" 未连接。确认模拟器处于对应演练模式且接口就绪后，加 --connect 运行。")
39         return 0
40     if not cs.dh or not cs.dh.strip():
41         parser.error(" 连接模拟器时必须通过 --robot-id 输入当前登录的队号")
42     mulu = cs.output / f"{cs.lx}_q{cs.problem}_{datetime.now():%Y%m%d_%H%M%S_%f}"
43     mulu.mkdir(parents=True)
44     xx = dict(
45         problem=cs.problem,
46         case_code=cs.al,
47         run_kind=cs.lx,
48         robot_id=cs.dh,
49         base_url=cs.url,
50         note=" 题号、测试类型和案例编码由操作者提供；接口不返回这些信息。",
51     )
52     (mulu / "metadata.json").write_text(
53         json.dumps(xx, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
54     )
55
56 def jd(sj):
57     if sj["event"] == "cleared":
58         print(
59             f"已清除频道 {sj['channel']}，累计 {sj['cleared_count']} 个，虚拟时间
60             ↪ {sj['virtual_time_s']:.3f} 秒",
61             flush=True,
62         )
```

```

62
63     with (mulu / "actions.jsonl").open("x", encoding="utf-8") as rizhi:
64         jk = xjzk(cs.dh, rizhi, cs.url)
65         zt = solver.xjzt(jk, cs.problem, jd, ghsj=cs.ghsj, dzys=cs.dzys)
66         jieguo = solver.yunxing(zt)
67         jieguo.update(xx)
68         (mulu / "summary.json").write_text(
69             json.dumps(jieguo, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
70         )
71     print(json.dumps(jieguo, ensure_ascii=False, indent=2))
72     print(f" 本地记录: {mulu}")
73     return 0 if zt["completed"] and zt["exit_confirmed"] else 2
74
75
76 if __name__ == "__main__":
77     sys.exit(main())

```

solve_questions.py

```

1 from pathlib import Path as lj
2 import sys
3
4 sys.path.insert(0, str(lj(__file__).resolve().parent / "src"))
5 from jammer_solver.questions import main
6
7 if __name__ == "__main__":
8     main()

```

start_robot.ps1

```

1 param(
2     [ValidateSet(3,4)][int]$problem = 4,
3     [ValidateSet('rehearsal','formal')][string]$runkind = 'rehearsal',
4     [string]$casetcode,
5     [string]$robotid,
6     [string]$python = 'python'
7 )
8 $ErrorActionPreference = 'Stop'
9 $runtime = (Get-Command $python -CommandType Application -ErrorAction Stop).Source
10 if ([string]::IsNullOrEmpty($casetcode)) { $casetcode = Read-Host 'Case code shown in
    ↳ simulator' }
11 if ([string]::IsNullOrEmpty($casetcode)) { throw 'Case code required' }
12 if ([string]::IsNullOrEmpty($robotid)) { $robotid = Read-Host 'Team ID currently logged into
    ↳ simulator' }
13 if ([string]::IsNullOrEmpty($robotid)) { throw 'Team ID required at runtime' }
14 Write-Host "Question=$problem Run=$runkind Team=$robotid Case=$casetcode"
15 Write-Host 'Start the matching simulator module yourself. This script does not start it.'
16 $confirmation = Read-Host 'Type CONNECT to send robot commands to the ready local interface'
17 if ($confirmation -cne 'CONNECT') { Write-Host 'No connection made.'; exit 0 }
18 & $runtime (Join-Path $PSScriptRoot 'run_robot.py') --problem $problem --run-kind $runkind
    ↳ --case-code $casetcode --robot-id $robotid --connect
19 exit $LASTEXITCODE

```

prepare_submission.py

```

1 """ 打包源程序、论文材料和正式日志。"""
2
3 import csv
4 from pathlib import Path as lj
5 import zipfile
6
7 root = lj(__file__).resolve().parent
8
9
10 def main():
11     output = root / "submission"
12     output.mkdir(exist_ok=True)

```

```

13 files = [
14     root / n
15     for n in (
16         "README.md",
17         "run_robot.py",
18         "solve_questions.py",
19         "start_robot.ps1",
20         "prepare_submission.py",
21         "AI 工具使用详情.pdf",
22     )
23 ]
24 for folder in ("src", "docs", "paper", "submission_inputs"):
25     files.extend(
26         p
27         for p in sorted((root / folder).rglob("*"))
28         if p.is_file()
29         and not p.is_symlink()
30         and not any(
31             x.startswith(".") or x == "__pycache__"
32             for x in p.relative_to(root).parts
33         )
34         and p.suffix in (".py", ".md", ".json", ".csv", ".tex", ".pdf", ".jlog")
35     )
36 files.extend((root / "submission_inputs").rglob(".gitkeep"))
37 with zipfile.ZipFile(
38     output / " 支撑材料.zip", "w", zipfile.ZIP_DEFLATED, compresslevel=9
39 ) as archive:
40     for p in sorted(files):
41         item = zipfile.ZipInfo(
42             p.relative_to(root).as_posix(), date_time=(2026, 1, 1, 0, 0, 0)
43         )
44         item.external_attr = 0o100644 << 16
45         archive.writestr(
46             item,
47             p.read_bytes(),
48             compress_type=zipfile.ZIP_DEFLATED,
49             compresslevel=9,
50         )
51 pending = []
52 with (root / "submission_inputs/formal_results.csv").open(
53     encoding="utf-8-sig", newline=""
54 ) as f:
55     for row in csv.DictReader(f):
56         name = f" 问题 {row['problem']} 第 {row['test']} 次"
57         folder = (
58             root / f"submission_inputs/formal/q{row['problem']}/test_{row['test']}"
59         )
60         if not any(folder.glob("*.jlog")):
61             pending.append(f"{name}: 补入原始正式日志")
62         if any(
63             not row.get(k, "").strip()
64             for k in (
65                 "case_code",
66                 "cleared_count",
67                 "virtual_time_s",
68                 "real_time_s",
69                 "log_filename",
70             )
71         ):
72             pending.append(f"{name}: 补填正式成绩表")
73 lines = ["# 待补材料", "", " 打包完成不表示缺少的数据已经补齐。", ""]
74 lines += [f"- [ ] {x}" for x in pending]
75 lines += [
76     "- [ ] 按实际结果补充正文、图表中的空栏。",
77     "- [ ] 据实填写 AI 工具使用详情中的待填内容。",
78     "- [ ] 保留官方日志的原始文件名和原始内容。",
79     "",
80     " 修改论文或源码后，先编译论文，再重新打包。",

```

```

81         " 生成提交 MD5 前备份最终 PDF 和 ZIP; 生成后保持这两个文件不变。",
82     ]
83     (output / " 待补清单.md").write_text("\n".join(lines) + "\n", encoding="utf-8")
84     print(" 已生成 submission/支撑材料.zip 和 submission/待补清单.md")
85
86
87 if __name__ == "__main__":
88     main()

```

src/jammer_solver/__init__.py

该文件为空，用于标记 Python 包。

src/jammer_solver/exact_geometry.py

```

1 from fractions import Fraction as fs
2 from functools import lru_cache as cache
3 import math
4
5 pilo = fs("3.14159265358979323846264338327950288419716939937510")
6 pihi = pilo + fs(1, 10**50)
7 eps = fs(201, 200)
8 domain = fs(1800)
9
10
11 def point(p):
12     if len(p) != 2 or not all((math.isfinite(float(x)) for x in p)):
13         raise ValueError("Non-finite point")
14     return tuple((fs(x) for x in p))
15
16
17 def jlxs(a, b, scale=1000):
18     v = ((a[0] - b[0]) * (a[0] - b[0]) + (a[1] - b[1]) * (a[1] - b[1])) * scale * scale
19     k = math.isqrt(v.numerator // v.denominator)
20     if fs(k * k) < v:
21         k += 1
22     return fs(k, scale)
23
24
25 def juxing(x0, y0, x1, y1):
26     return [point(p) for p in ((x0, y0), (x1, y0), (x1, y1), (x0, y1))]
27
28
29 def hull(points):
30     pts = sorted(set(points))
31     if len(pts) <= 1:
32         return pts
33     lo, hi = ([], [])
34     for p in pts:
35         while (
36             len(lo) > 1
37             and (lo[-1][0] - lo[-2][0]) * (p[1] - lo[-1][1])
38             - (lo[-1][1] - lo[-2][1]) * (p[0] - lo[-1][0])
39             <= 0
40         ):
41             lo.pop()
42         lo.append(p)
43     for p in reversed(pts):
44         while (
45             len(hi) > 1
46             and (hi[-1][0] - hi[-2][0]) * (p[1] - hi[-1][1])
47             - (hi[-1][1] - hi[-2][1]) * (p[0] - hi[-1][0])
48             <= 0
49         ):
50             hi.pop()
51         hi.append(p)
52     return lo[:-1] + hi[:-1]
53

```

```

54
55 def clip(poly, normal, bound):
56     if not poly:
57         return []
58     out = []
59     a = poly[-1]
60     fa = normal[0] * a[0] + normal[1] * a[1] - bound
61     for b in poly:
62         fb = normal[0] * b[0] + normal[1] * b[1] - bound
63         if (fa > 0) != (fb > 0):
64             t = fa / (fa - fb)
65             out.append((a[0] + t * (b[0] - a[0]), a[1] + t * (b[1] - a[1])))
66         if fb <= 0:
67             out.append(b)
68         a, fa = (b, fb)
69     result = []
70     for p in out:
71         if not result or result[-1] != p:
72             result.append(p)
73     if len(result) > 1 and result[0] == result[-1]:
74         result.pop()
75     return result
76
77
78 def contains(poly, p):
79     if not poly:
80         return False
81     if len(poly) == 1:
82         return p == poly[0]
83     if len(poly) == 2:
84         return (poly[1][0] - poly[0][0]) * (p[1] - poly[0][1]) - (
85             poly[1][1] - poly[0][1]
86             ) * (p[0] - poly[0][0]) == 0 and (p[0] - poly[0][0]) * (p[0] - poly[1][0]) + (
87             p[1] - poly[0][1]
88             ) * (p[1] - poly[1][1]) <= 0
89     return all(
90         (
91             (b[0] - a[0]) * (p[1] - a[1]) - (b[1] - a[1]) * (p[0] - a[0]) >= 0
92             for a, b in zip(poly, poly[1:] + poly[:1])
93         )
94     )
95
96
97 @cache(maxsize=4096)
98 def trig(degrees):
99     degrees = fs(degrees)
100     xq = int((degrees + 45) // 90)
101     d = degrees - 90 * xq
102     x = d * (pilo + pihi) / 360
103     err = abs(d) * (pihi - pilo) / 360
104     sin = sum(
105         ((-1) ** j * x ** (2 * j + 1) / math.factorial(2 * j + 1) for j in range(10)),
106         fs(0),
107     )
108     cos = sum(
109         ((-1) ** j * x ** (2 * j) / math.factorial(2 * j) for j in range(10)), fs(0)
110     )
111     se = fs(1, math.factorial(20)) + err
112     ce = fs(1, math.factorial(19)) + err
113     c, s = ((cos - ce, cos + ce), (sin - se, sin + se))
114     for _ in range(xq % 4):
115         c, s = ((-s[1], -s[0]), c)
116     scale = 10**12
117
118 def outward(pair):
119     a, b = pair
120     return (fs(a * scale // 1, scale), fs(-b * scale // 1, scale))
121

```

```

122     return (*outward(c), *outward(s))
123
124
125 def fangxiang(degrees):
126     cl, ch, sl, sh = trig(fs(degrees))
127     return (((cl + ch) / 2, (sl + sh) / 2), ((ch - cl) / 2, (sh - sl) / 2))
128
129
130 def bqdpmpoly(poly, origin, normal, error, rhs=fs(0)):
131     slack = error[0] * (domain + abs(origin[0])) + error[1] * (domain + abs(origin[1]))
132     return clip(
133         poly, normal, normal[0] * origin[0] + normal[1] * origin[1] + rhs + slack
134     )
135
136
137 @cache(maxsize=1)
138 def yfw():
139     poly = juxing(-1800, -1800, 1800, 1800)
140     for angle in range(0, 360, 15):
141         u, e = fangxiang(fs(angle))
142         poly = bqdpmpoly(poly, point((0, 0)), u, e, fs(1800))
143     return tuple(poly)
144
145
146 def fwys(lo, hi=None):
147     hi = lo if hi is None else hi
148     if fs(hi) - fs(lo) + 2 * eps >= 180:
149         return []
150     ulo, elo = fangxiang(fs(lo) - eps)
151     uhi, ehi = fangxiang(fs(hi) + eps)
152     return [
153         ((ulo[1], -ulo[0]), (elo[1], elo[0]), fs(0)),
154         ((-uhi[1], uhi[0]), (ehi[1], ehi[0]), fs(0)),
155     ]
156
157
158 def jcfw(poly, origin, bearing, radius=fs(1500), hi=None):
159     ys = fwys(bearing, hi)
160     if hi is None:
161         u, e = fangxiang(fs(bearing))
162         ce = trig(eps)[0]
163         ys += [(u, e, radius), ((-u[0], -u[1]), e, -5 * ce)]
164     for n, e, b in ys:
165         poly = bqdpmpoly(poly, origin, n, e, b)
166     for n, b in (
167         ((1, 0), origin[0] + 1500),
168         ((-1, 0), 1500 - origin[0]),
169         ((0, 1), origin[1] + 1500),
170         ((0, -1), 1500 - origin[1]),
171     ):
172         poly = clip(poly, n, b)
173     return poly
174
175
176 def xzqj(origin, bearing, bounds):
177     u, e = fangxiang(fs(bearing))
178     corners = juxing(*bounds)
179     out = []
180     for x, y in corners:
181         center = (origin[0] + u[0] * x - u[1] * y, origin[1] + u[1] * x + u[0] * y)
182         dx, dy = (e[0] * abs(x) + e[1] * abs(y), e[1] * abs(x) + e[0] * abs(y))
183         out += juxing(center[0] - dx, center[1] - dy, center[0] + dx, center[1] + dy)
184     return hull(out)
185
186
187 def xzd(origin, bearing, x, y):
188     u, e = fangxiang(fs(bearing))
189     mid = (origin[0] + u[0] * x - u[1] * y, origin[1] + u[1] * x + u[0] * y)

```

```

190     p = point(tuple((float(x) for x in mid)))
191     dx = abs(p[0] - mid[0]) + e[0] * abs(x) + e[1] * abs(y)
192     dy = abs(p[1] - mid[1]) + e[1] * abs(x) + e[0] * abs(y)
193     if dx * dx + dy * dy > 1:
194         raise ArithmeticError("Rotation exceeds the 1 m allowance")
195     return p
196
197
198 def ypbh(poly, center, radius):
199     return bool(poly) and all(
200         (
201             (v[0] - center[0]) * (v[0] - center[0])
202             + (v[1] - center[1]) * (v[1] - center[1])
203             <= radius * radius
204             for v in poly
205         )
206     )
207
208
209 def syxc(poly, pos, a, b):
210     va, vb = ((a[0] - pos[0], a[1] - pos[1]), (b[0] - pos[0], b[1] - pos[1]))
211     if va[0] * vb[1] - va[1] * vb[0] < 0:
212         a, b, va, vb = (b, a, vb, va)
213     if va[0] * vb[1] - va[1] * vb[0] == 0:
214         return (poly, False)
215     edge = (b[0] - a[0], b[1] - a[1])
216     normal = (edge[1], -edge[0])
217     bound = normal[0] * a[0] + normal[1] * a[1]
218     if normal[0] * pos[0] + normal[1] * pos[1] > bound:
219         normal = (-normal[0], -normal[1])
220         bound = -bound
221     outside = clip(poly, (-normal[0], -normal[1]), -bound)
222     if not outside:
223         return (poly, False)
224     if not all(
225         (
226             va[0] * (s[1] - pos[1]) - va[1] * (s[0] - pos[0]) >= 0
227             and (s[0] - pos[0]) * vb[1] - (s[1] - pos[1]) * vb[0] >= 0
228             for s in outside
229         )
230     ):
231         return (poly, False)
232     if not ypbh(outside, a, fs(1000)) or not ypbh(outside, b, fs(1000)):
233         return (poly, False)
234     result = clip(poly, normal, bound)
235     return (result, result != poly)
236
237
238 def sswd():
239     return [
240         point((600 * i, 600 * j))
241         for j in range(-3, 4)
242         for i in (range(-3, 4) if j % 2 else range(3, -4, -1))
243     ]
244
245
246 @cache(maxsize=256)
247 def fgpd(radius, optical, problem, jdmax=2048, sdmax=12):
248     radio, optical = (tuple(map(point, radio)), tuple(map(point, optical)))
249     stack = [(juxing(-1800, -1800, 1800, 1800), 0)]
250     nodes = 0
251     rf = [(q, tuple((float(x) for x in q))) for q in radio]
252     while stack and nodes < jdmax:
253         box, depth = stack.pop()
254         nodes += 1
255         x0, y0 = box[0]
256         x1, y1 = box[2]
257         dx, dy = (max(fs(0), x0, -x1), max(fs(0), y0, -y1))

```

```

258     if dx * dx + dy * dy > domain**2:
259         continue
260     hit = next((q for q in optical if ypbh(box, q, fs(20))), None)
261     if hit is not None:
262         continue
263     bf = [tuple((float(x) for x in v)) for v in box]
264     nearby = [
265         q
266         for q, qf in rf
267         if all((math.dist(qf, v) < 1000.000001 for v in bf))
268         and ypbh(box, q, fs(1000))
269     ]
270     if problem == 3 and nearby:
271         continue
272     if problem == 4 and len(nearby) >= 3:
273         h = hull(nearby)
274         if all((contains(h, v) for v in box)):
275             continue
276     if depth >= sdmax:
277         return False
278     mx, my = ((x0 + x1) / 2, (y0 + y1) / 2)
279     stack.extend(
280         (
281             (juxing(*v), depth + 1)
282             for v in (
283                 (x0, y0, mx, my),
284                 (mx, y0, x1, my),
285                 (x0, my, mx, y1),
286                 (mx, my, x1, y1),
287             )
288         )
289     )
290     return not stack

```

src/jammer_solver/protocol.py

```

1 import json
2 import math
3 import os
4 import time
5 import uuid
6 from urllib.request import Request as qqcls, ProxyHandler as dlcls, build_opener as ljg
7
8
9 def jyh(path, reply):
10     if not isinstance(reply, dict) or type(reply.get("accepted")) is not bool:
11         raise RuntimeError("Malformed acceptance status")
12     if not reply["accepted"]:
13         raise RuntimeError("accepted=false: action did not execute")
14     if (
15         not (
16             type(reply.get("virtual_time_s")) in (int, float)
17             and math.isfinite(reply.get("virtual_time_s"))
18         )
19         or reply["virtual_time_s"] < 0
20     ):
21         raise RuntimeError("Malformed virtual time in accepted response")
22     if path == "/enter":
23         seconds = reply.get("remaining_real_duration_s")
24         if (
25             not (type(seconds) in (int, float) and math.isfinite(seconds))
26             or not 0 <= seconds <= 1200
27             or int(seconds) != seconds
28         ):
29             raise RuntimeError("Malformed remaining real duration")
30     if (
31         not (
32             type(reply.get("max_virtual_duration_s")) in (int, float)

```

```

33         and math.isfinite(reply.get("max_virtual_duration_s"))
34     )
35     or reply["max_virtual_duration_s"] <= 0
36 ):
37     raise RuntimeError("Malformed maximum virtual duration")
38 elif path == "/measure":
39     result = reply.get("measure_result")
40     if result not in ("direction", "near", "no_signal"):
41         raise RuntimeError("Unknown measure result")
42     if result == "direction":
43         angle = reply.get("svd_deg")
44         if (
45             not (type(angle) in (int, float) and math.isfinite(angle))
46             or not 0 <= angle < 360
47         ):
48             raise RuntimeError("Malformed bearing")
49 elif path == "/clear" and reply.get("clear_result") not in (
50     "success",
51     "no_target_in_range",
52 ):
53     raise RuntimeError("Unknown clear result")
54 elif path == "/exit" and reply.get("exit_reason") != "user_exit":
55     raise RuntimeError("Unknown exit result")
56
57
58 def xjkk(dh, rizhi, url="http://127.0.0.1:2026"):
59     jiekou = dict(pending=None, deadline=math.inf)
60     bianhao = uuid.uuid4().hex[:16]
61     xulie = 0
62     opener = ljq(dlccls({}))
63
64     def baocun(shijian):
65         rizhi.write(json.dumps(shijian, ensure_ascii=False, allow_nan=False) + "\n")
66         rizhi.flush()
67         os.fsync(rizhi.fileno())
68
69     def qingqiu(path, weizhi=None, pindao=None):
70         nonlocal xulie
71         if jiekou["pending"] is not None:
72             raise RuntimeError("Previous request has no confirmed response")
73         shengyu = jiekou["deadline"] - time.monotonic()
74         if shengyu <= 0:
75             raise TimeoutError("Request deadline reached")
76         xulie += 1
77         neirong = dict(arena_id="default", robot_id=dh, request_id=f"{bianhao}-{xulie}")
78         if path in ("/measure", "/clear"):
79             neirong.update(position=dict(x=weizhi[0], y=weizhi[1]), channel=pindao)
80         zhengwen = json.dumps(
81             neirong, ensure_ascii=False, allow_nan=False, separators=(",", ":")
82         ).encode("utf-8")
83         baocun(dict(event="intent", wall_time=time.time(), path=path, payload=neirong))
84         jiekou["pending"] = dict(path=path, payload=neirong)
85         request = qqcls(
86             url + path,
87             data=zhengwen,
88             headers={"Content-Type": "application/json"},
89             method="POST",
90         )
91         with opener.open(request, timeout=min(5.0, shengyu)) as response:
92             huifu = json.loads(response.read().decode("utf-8"))
93             baocun(
94                 dict(
95                     event="response",
96                     wall_time=time.time(),
97                     request_id=neirong["request_id"],
98                     attempt=0,
99                     http_status=response.status,
100                     response=huifu,

```

```

101         )
102     )
103     if response.status != 200:
104         raise RuntimeError(f"HTTP {response.status}")
105     jyhfh(path, huifu)
106     jiekou["pending"] = None
107     return huifu
108
109     jiekou.update(call=qingqiu, record=baocun)
110     return jiekou

```

src/jammer_solver/questions.py

```

1 import argparse
2 import heapq
3 import json
4 import math
5 from itertools import combinations
6 from pathlib import Path as lj
7
8 eps = math.radians(1.005)
9
10
11 def hull(points):
12     points = sorted(set(points))
13     if len(points) <= 1:
14         return points
15
16     def chain(seq):
17         out = []
18         for p in seq:
19             while (
20                 len(out) >= 2
21                 and (out[-1][0] - out[-2][0]) * (p[1] - out[-1][1])
22                   - (out[-1][1] - out[-2][1]) * (p[0] - out[-1][0])
23                 <= 0
24             ):
25                 out.pop()
26             out.append(p)
27         return out
28
29     return chain(points)[-1] + chain(points[::-1])[-1]
30
31
32 def zydd(vertices):
33     n = len(vertices)
34     if n == 1:
35         return (vertices[0], vertices[0])
36     if n == 2:
37         return tuple(vertices)
38     best = (vertices[0], vertices[1])
39     bestd = math.dist(*best)
40     j = 1
41     for i in range(n):
42         nxt = (i + 1) % n
43         edge = (vertices[nxt][0] - vertices[i][0], vertices[nxt][1] - vertices[i][1])
44
45         def area(k):
46             return abs(
47                 edge[0] * (vertices[k][1] - vertices[i][1])
48                 - edge[1] * (vertices[k][0] - vertices[i][0])
49             )
50
51         steps = 0
52         while area((j + 1) % n) > area(j) and steps < n:
53             j = (j + 1) % n
54             steps += 1
55         for a in (i, nxt):

```

```

56         for b in (j, (j + 1) % n):
57             d = math.dist(vertices[a], vertices[b])
58             if d > bestd:
59                 best, bestd = ((vertices[a], vertices[b]), d)
60     return best
61
62
63 def jhqy(gclb, wcjd=1.005):
64     planes = []
65     for obs in gclb:
66         x, y, angle = (float(obs[k]) for k in ("x", "y", "bearing_deg"))
67         if not all((math.isfinite(v) for v in (x, y, angle))):
68             raise ValueError("Observations must be finite")
69         lo, hi = (math.radians(angle - wcjd), math.radians(angle + wcjd))
70         for nx, ny in ((math.sin(lo), -math.cos(lo)), (-math.sin(hi), math.cos(hi))):
71             planes.append((nx, ny, nx * x + ny * y))
72     base = dict(
73         epsilon_deg=wcjd,
74         diameter=None,
75         vertices=[],
76         numerical_method="float64; relative feasibility tolerance 1e-10",
77     )
78     if not planes:
79         return dict(base, state="UNBOUNDED")
80
81     def feasible(x, y):
82         return all(
83             (
84                 a * x + b * y - c <= 1e-10 * (1 + abs(a * x) + abs(b * y) + abs(c))
85                 for a, b, c in planes
86             )
87         )
88
89     bx = []
90     for (a, b, c), (d, e, f) in combinations(planes, 2):
91         det = a * e - b * d
92         if abs(det) <= 1e-14:
93             continue
94         x, y = ((c * e - b * f) / det, (a * f - c * d) / det)
95         if not math.isfinite(x) or not math.isfinite(y):
96             return dict(base, state="NUMERICALLY_UNRESOLVED")
97         if feasible(x, y):
98             bx.append((x, y))
99     if not bx:
100         return dict(base, state="EMPTY_OR_NUMERICALLY_UNRESOLVED")
101     for a, b, _ in planes:
102         for dx, dy in ((b, -a), (-b, a)):
103             if all((u * dx + v * dy <= 1e-14 for u, v, _ in planes)):
104                 return dict(base, state="UNBOUNDED", feasible_point=bx[0])
105     vertices = hull(bx)
106     p, q = zydd(vertices)
107     diameter = math.dist(p, q)
108     center = ((p[0] + q[0]) / 2, (p[1] + q[1]) / 2)
109     radius = diameter / 2
110     return dict(
111         base,
112         state={1: "POINT", 2: "SEGMENT"}.get(len(vertices), "POLYGON"),
113         vertices=vertices,
114         diameter=diameter,
115         farthest_pair=(p, q),
116         diameter_circle_center=center,
117         diameter_circle_radius=radius,
118         diameter_circle_covers=all(
119             (math.dist(v, center) <= radius + 1e-09 * (1 + radius) for v in vertices)
120         ),
121     )
122
123

```

```

124 def ecdqy(first, bearing, query=None):
125     centers = [tuple(first)]
126     for sign in (1, -1):
127         angle = math.radians(bearing) + sign * eps
128         centers.append(
129             (first[0] + 1000 * math.cos(angle), first[1] + 1000 * math.sin(angle))
130         )
131     normal = (math.cos(math.radians(bearing)), math.sin(math.radians(bearing)))
132     result = dict(
133         disk_centers=centers,
134         disk_radii=[1000] * 3,
135         forward_halfplane=dict(
136             normal=normal, origin=first, relation="dot(normal, q-origin) >= 0"
137         ),
138         epsilon_deg=1.005,
139         scope="Exact for relaxed radial prior [0,1500] within forward halfplane; conservative for
            ↪ actual r>5 and source domain",
140     )
141     if query is not None:
142         if len(query) != 2 or not all((math.isfinite(v) for v in query)):
143             raise ValueError("Query must contain two finite coordinates")
144         result["query_in_region"] = (
145             all((math.hypot(query[0] - c[0], query[1] - c[1]) <= 1000 for c in centers))
146             and sum((normal[k] * (query[k] - first[k]) for k in (0, 1))) >= 0
147         )
148     return result
149
150
151 def zjsj(rho, theta):
152     a, b = (rho * math.cos(theta), rho * math.sin(theta))
153     dist = max(
154         (
155             math.sqrt(r * r + rho * rho - 2 * r * rho * math.cos(theta + eps))
156             for r in (5.0, 1500.0)
157         )
158     )
159     height = b * math.cos(eps) - a * math.sin(eps)
160     return (djsj(dist, height), dist)
161
162
163 def djsj(dist, height):
164     sine = max(0.0, min(1.0, height / max(dist, 1e-12)))
165     gamma = math.asin(sine) - 2 * eps
166     if gamma <= 0:
167         return math.inf
168     w1, w2 = (1500 * math.sin(eps), dist * math.sin(eps))
169     return (
170         2
171         * math.sqrt(w1 * w1 + w2 * w2 + 2 * w1 * w2 * math.cos(gamma))
172         / math.sin(gamma)
173     )
174
175
176 def jdyh(rho, tol=0.02, jdmax=1024):
177     if not 5 < rho < 1000:
178         raise ValueError("A positive reception margin requires 5 < movement < 1000")
179     lo = eps + 1e-08
180     hi = math.acos(rho / 2000) - eps - 1e-08
181
182     def lower(a, b):
183         d = zjsj(rho, a)[1]
184         h = rho * math.sin(b - eps)
185         return max(0.0, djsj(d, h) - 1e-06)
186
187     best = min(((zjsj(rho, t)[0], t) for t in (lo, (lo + hi) / 2, hi)))
188     heap = [(lower(lo, hi), lo, hi)]
189     nodes = 0
190     prunedlb = math.inf

```

```

191 while heap and nodes < jdmax:
192     lb, a, b = heapq.heappop(heap)
193     if best[0] - lb <= tol:
194         prunelb = min(prunelb, lb)
195         break
196     mid = (a + b) / 2
197     for x, y in ((a, mid), (mid, b)):
198         t = (x + y) / 2
199         value = zjsj(rho, t)[0]
200         nodes += 1
201         if value < best[0]:
202             best = (value, t)
203             bound = lower(x, y)
204             if bound < best[0]:
205                 heapq.heappush(heap, (bound, x, y))
206         else:
207             prunelb = min(prunelb, bound)
208 boundlo = min(best[0], prunelb, min((entry[0] for entry in heap), default=math.inf))
209 bound, theta = best
210 a, b = (rho * math.cos(theta), rho * math.sin(theta))
211 endpos = max(
212     (
213         math.dist((a, b), (1000 * math.cos(eps), s * 1000 * math.sin(eps)))
214         for s in (-1, 1)
215     )
216 )
217 return dict(
218     movement_m=rho,
219     a_m=a,
220     b_m=b,
221     diameter_upper_m=bound,
222     angle_family_lower_m=boundlo,
223     optimization_gap_m=bound - boundlo,
224     reception_margin_m=1000 - max(rho, endpos),
225     nodes=nodes,
226     scope="entire_feasible_angle_interval_at_fixed_movement_radius",
227 )
228
229
230 def zxdj(target=163.0, jdmax=6000, tol=0.1):
231     seed = jdyh(900.0)
232     if seed["diameter_upper_m"] > target:
233         seed = jdyh(999.0)
234     if seed["diameter_upper_m"] > target:
235         raise ValueError(
236             "No initial feasible design for this target; choose a larger diameter target"
237         )
238     best = (
239         seed["movement_m"],
240         math.atan2(seed["b_m"], seed["a_m"]),
241         seed["diameter_upper_m"],
242     )
243     heap = []
244     serial = 0
245     nodes = 0
246
247     def add(r0, r1, t0, t1):
248         nonlocal serial
249         r1 = min(r1, 2000 * math.cos(t0 + eps))
250         t1 = min(t1, math.acos(r0 / 2000) - eps)
251         if r0 >= r1 or t0 >= t1 or r0 >= best[0]:
252             return
253         dists = []
254         for r in (5.0, 1500.0):
255             rho = max(r0, min(r1, r * math.cos(t0 + eps)))
256             dists.append(
257                 math.sqrt(
258                     max(0.0, r * r + rho * rho - 2 * r * rho * math.cos(t0 + eps))

```

```

259         )
260     )
261     lb = djsj(max(dists), r1 * math.sin(t1 - eps)) - 1e-06
262     if lb > target:
263         return
264     serial += 1
265     heapq.heappush(heap, (r0, serial, (r0, r1, t0, t1)))
266
267     add(5.0001, best[0], eps + 1e-08, math.pi / 2 - eps - 1e-08)
268     stoplb = None
269     while heap and nodes < jdmax:
270         lower, _, (r0, r1, t0, t1) = heapq.heappop(heap)
271         if best[0] - lower <= tol:
272             stoplb = lower
273             break
274         if lower >= best[0]:
275             continue
276         nodes += 1
277         rm = (r0 + r1) / 2
278         tm = (t0 + t1) / 2
279         for rho, theta in ((rm, tm), (r1, tm), (rm, min(t1, max(t0, best[1])))):
280             if rho < best[0] and rho < 1000 and (theta <= math.acos(rho / 2000) - eps):
281                 bound = zjsj(rho, theta)[0]
282                 if bound <= target - 1e-05:
283                     best = (rho, theta, bound)
284             if (r1 - r0) / 1000 > (t1 - t0) / (math.pi / 2):
285                 add(r0, rm, t0, t1)
286                 add(rm, r1, t0, t1)
287             else:
288                 add(r0, r1, t0, tm)
289                 add(r0, r1, tm, t1)
290         lower = min(
291             [best[0]] + ([stoplb] if stoplb is not None else []) + [x[0] for x in heap]
292         )
293         rho, theta, bound = best
294         a, b = (rho * math.cos(theta), rho * math.sin(theta))
295         return dict(
296             movement_m=rho,
297             a_m=a,
298             b_m=b,
299             diameter_upper_m=bound,
300             target_diameter_m=target,
301             movement_lower_m=lower,
302             movement_gap_m=rho - lower,
303             nodes=nodes,
304             reception_margin_m=1000
305             - max(
306                 rho,
307                 max(
308                     (
309                         math.dist((a, b), (1000 * math.cos(eps), s * 1000 * math.sin(eps)))
310                         for s in (-1, 1)
311                     )
312                 ),
313             ),
314             scope="global_movement_bound_for_the_analytic_strip_criterion_not_exact_physical_diamete_
↪ r",
315         )
316
317
318 def main():
319     parser = argparse.ArgumentParser(description=" 前两问的离线几何计算；不连接任何接口")
320     parser.add_argument(
321         "input", type=lj, help="JSON: observations, 或 first 与 bearing_deg"
322     )
323     parser.add_argument("--problem", type=int, choices=(1, 2), required=True)
324     parser.add_argument("--output", type=lj)
325     args = parser.parse_args()

```

```

326 data = json.loads(args.input.read_text(encoding="utf-8-sig"))
327 if args.problem == 1:
328     result = jhqy(data["observations"], data.get("epsilon_deg", 1.005))
329 else:
330     design = (
331         jdyh(data["movement_m"])
332         if "movement_m" in data
333         else zxdj(data.get("target_diameter_m", 163.0))
334     )
335     theta = math.radians(data["bearing_deg"])
336     u = (math.cos(theta), math.sin(theta))
337     v = (-u[1], u[0])
338     options = [
339         tuple(
340             (
341                 data["first"][i]
342                 + design["a_m"] * u[i]
343                 + sign * design["b_m"] * v[i]
344                 for i in (0, 1)
345             )
346         )
347         for sign in (1, -1)
348     ]
349     p = (
350         min(options, key=lambda p: math.dist(p, data["current"]))
351         if "current" in data
352         else options[0]
353     )
354     result = dict(
355         second_position=dict(x=p[0], y=p[1]),
356         candidate_region=ecdqy(
357             tuple(data["first"]), data["bearing_deg"], data.get("query")
358         ),
359         guaranteed_diameter_bound_m=design["diameter_upper_m"],
360         optimization=design,
361         selection_status="Minimum movement for the requested analytic diameter bound;
362         ↪ remaining radius gap is reported"
363         if "movement_m" not in data
364         else "Analytic diameter bound optimized over the full feasible angle interval at the
365         ↪ chosen movement radius",
366         rule="movement_m is measured from first; nearest symmetric side if current supplied",
367         scope=" 全向信号; 首次方向有效; 半径 1000 至 1500 米",
368     )
369 text = json.dumps(result, ensure_ascii=False, indent=2, allow_nan=False)
370 if args.output:
371     args.output.write_text(text + "\n", encoding="utf-8")
372 print(text)

```

src/jammer_solver/solver.py

```

1 from fractions import Fraction as fs
2 from itertools import combinations
3 import math
4 import time
5 from . import exact_geometry as g
6
7 qcbj = 19.8
8
9
10 def lxcd(start, points, end=None):
11     path = [start] + list(points) + ([[] if end is None else [end]])
12     return sum((math.dist(a, b) for a, b in zip(path, path[1:])))
13
14
15 def lxpx(start, points, end=None):
16     if len(points) < 2:
17         return list(points)
18     rest = list(points)

```

```

19 greedy = []
20 p = start
21 while rest:
22     q = min(rest, key=lambda q: (math.dist(p, q), q))
23     greedy.append(q)
24     rest.remove(q)
25     p = q
26 order = min((list(points), greedy), key=lambda qs: lxcd(start, qs, end))
27 for _ in range(8):
28     bestdelta, bestpair = (0.0, None)
29     for i in range(len(order) - 1):
30         a = start if i == 0 else order[i - 1]
31         for j in range(i + 1, len(order)):
32             b = order[j + 1] if j + 1 < len(order) else end
33             before = math.dist(a, order[i])
34             after = math.dist(a, order[j])
35             if b is not None:
36                 before += math.dist(order[j], b)
37                 after += math.dist(order[i], b)
38             if after - before < bestdelta - 1e-08:
39                 bestdelta, bestpair = (after - before, (i, j))
40     if bestpair is None:
41         break
42     i, j = bestpair
43     order[i : j + 1] = reversed(order[i : j + 1])
44 return order
45
46
47 def zjqcd(polygon, current):
48     if not polygon:
49         return None
50     radius = qcbj - 1e-06
51
52     def feasible(p):
53         return all(
54             (math.hypot(p[0] - v[0], p[1] - v[1]) <= radius + 1e-10 for v in polygon)
55         )
56
57     if feasible(current):
58         return current
59     if any(
60         (
61             math.hypot(a[0] - b[0], a[1] - b[1]) > 2 * radius
62             for a, b in combinations(polygon, 2)
63         )
64     ):
65         return None
66     best, zxjl = (None, float("inf"))
67
68     def consider(p):
69         nonlocal best, zxjl
70         travel = math.hypot(current[0] - p[0], current[1] - p[1])
71         if travel < zxjl and feasible(p):
72             best, zxjl = (p, travel)
73
74     for a in polygon:
75         d = math.hypot(current[0] - a[0], current[1] - a[1])
76         if d > radius:
77             consider(
78                 (
79                     a[0] + radius * (current[0] - a[0]) / d,
80                     a[1] + radius * (current[1] - a[1]) / d,
81                 )
82             )
83     for a, b in combinations(polygon, 2):
84         d = math.hypot(a[0] - b[0], a[1] - b[1])
85         if d <= 1e-12 or d > 2 * radius:
86             continue

```

```

87     mid = ((a[0] + b[0]) / 2, (a[1] + b[1]) / 2)
88     h = math.sqrt(max(0.0, radius * radius - d * d / 4))
89     dx, dy = (-(b[1] - a[1]) / d * h, (b[0] - a[0]) / d * h)
90     consider((mid[0] + dx, mid[1] + dy))
91     consider((mid[0] - dx, mid[1] - dy))
92     return best
93
94
95 def qclxyh(centers, radius, start, current, target, bsmax=24):
96
97     def feasible(p):
98         return all(
99             (math.hypot(p[0] - c[0], p[1] - c[1]) <= radius + 1e-10 for c in centers)
100         )
101
102     if not centers or radius <= 0 or (not feasible(start)):
103         return (start, dict(iterations=0, gap_m=None))
104     corners = []
105     for a, b in combinations(centers, 2):
106         d = math.hypot(a[0] - b[0], a[1] - b[1])
107         if d <= 1e-12 or d > 2 * radius:
108             continue
109         mid = ((a[0] + b[0]) / 2, (a[1] + b[1]) / 2)
110         h = math.sqrt(max(0.0, radius * radius - d * d / 4))
111         dx, dy = (-(b[1] - a[1]) * h / d, (b[0] - a[0]) * h / d)
112         for p in ((mid[0] + dx, mid[1] + dy), (mid[0] - dx, mid[1] - dy)):
113             if feasible(p):
114                 corners.append(p)
115     point, steps, gap = (start, 0, None)
116     for _ in range(bsmax):
117         value = math.hypot(current[0] - point[0], current[1] - point[1]) + math.hypot(
118             point[0] - target[0], point[1] - target[1]
119         )
120         if value - math.hypot(current[0] - target[0], current[1] - target[1]) <= 1e-07:
121             gap = max(
122                 0.0, value - math.hypot(current[0] - target[0], current[1] - target[1])
123             )
124             break
125         d1, d2 = (
126             math.hypot(point[0] - current[0], point[1] - current[1]),
127             math.hypot(point[0] - target[0], point[1] - target[1]),
128         )
129         if min(d1, d2) <= 1e-12:
130             gap = 0.0
131             break
132         g = (
133             (point[0] - current[0]) / d1 + (point[0] - target[0]) / d2,
134             (point[1] - current[1]) / d1 + (point[1] - target[1]) / d2,
135         )
136         norm = math.hypot(*g)
137         if norm <= 1e-12:
138             gap = None
139             break
140         beixuan = corners + [point]
141         for c in centers:
142             cand = (c[0] - radius * g[0] / norm, c[1] - radius * g[1] / norm)
143             if feasible(cand):
144                 beixuan.append(cand)
145         support = min(beixuan, key=lambda p: g[0] * p[0] + g[1] * p[1])
146         gap = max(0.0, g[0] * (point[0] - support[0]) + g[1] * (point[1] - support[1]))
147         if gap <= 1e-05:
148             break
149         lo, hi = (0.0, 1.0)
150         for _ in range(36):
151             left, right = ((2 * lo + hi) / 3, (lo + 2 * hi) / 3)
152             lp = (
153                 point[0] + left * (support[0] - point[0]),
154                 point[1] + left * (support[1] - point[1]),

```

```

155         )
156         rp = (
157             point[0] + right * (support[0] - point[0]),
158             point[1] + right * (support[1] - point[1]),
159         )
160         if math.hypot(current[0] - lp[0], current[1] - lp[1]) + math.hypot(
161             lp[0] - target[0], lp[1] - target[1]
162         ) <= math.hypot(current[0] - rp[0], current[1] - rp[1]) + math.hypot(
163             rp[0] - target[0], rp[1] - target[1]
164         ):
165             hi = right
166         else:
167             lo = left
168     cand = min(
169         (
170             point,
171             support,
172             (
173                 point[0] + (lo + hi) / 2 * (support[0] - point[0]),
174                 point[1] + (lo + hi) / 2 * (support[1] - point[1]),
175             ),
176         ),
177         key=lambda p: (
178             math.hypot(current[0] - p[0], current[1] - p[1])
179             + math.hypot(p[0] - target[0], p[1] - target[1])
180         ),
181     )
182     if not feasible(cand):
183         break
184     gain = value - (
185         math.hypot(current[0] - cand[0], current[1] - cand[1])
186         + math.hypot(cand[0] - target[0], cand[1] - target[1])
187     )
188     point = cand
189     steps += 1
190     gap = None
191     if gain <= 1e-09:
192         break
193     return (point, dict(iterations=steps, gap_m=gap))
194
195
196 def zhipai(cost):
197     n = len(cost)
198     u = [0.0] * (n + 1)
199     v = [0.0] * (n + 1)
200     p = [0] * (n + 1)
201     way = [0] * (n + 1)
202     for i in range(1, n + 1):
203         p[0] = i
204         j0 = 0
205         minimum = [float("inf")] * (n + 1)
206         used = [False] * (n + 1)
207         while True:
208             used[j0] = True
209             i0 = p[j0]
210             delta = float("inf")
211             j1 = 0
212             for j in range(1, n + 1):
213                 if not used[j]:
214                     cur = cost[i0 - 1][j - 1] - u[i0] - v[j]
215                     if cur < minimum[j]:
216                         minimum[j] = cur
217                         way[j] = j0
218                     if minimum[j] < delta:
219                         delta = minimum[j]
220                         j1 = j
221             for j in range(n + 1):
222                 if used[j]:

```

```

223             u[p[j]] += delta
224             v[j] -= delta
225         else:
226             minimum[j] -= delta
227         j0 = j1
228         if p[j0] == 0:
229             break
230         while True:
231             j1 = way[j0]
232             p[j0] = p[j1]
233             j0 = j1
234             if j0 == 0:
235                 break
236         succ = [0] * n
237         for j in range(1, n + 1):
238             succ[p[j] - 1] = j - 1
239         return (sum((cost[i][j] for i, j in enumerate(succ))), succ)
240
241
242 def sscd(problem):
243     if problem == 3:
244         return [(0.0, 0.0)] + [
245             (1125 * math.cos(k * math.pi / 3), 1125 * math.sin(k * math.pi / 3))
246             for k in range(6)
247         ]
248     return (
249         [(0.0, 0.0)]
250         + [
251             (998 * math.cos(k * math.pi / 4), 998 * math.sin(k * math.pi / 4))
252             for k in range(8)
253         ]
254         + [
255             (
256                 1868 * math.cos(k * math.pi / 6 + math.pi / 12),
257                 1868 * math.sin(k * math.pi / 6 + math.pi / 12),
258             )
259             for k in range(12)
260         ]
261     )
262
263
264 def pindao(jilu, status):
265     return {c for c, s in jilu["channels"].items() if s["status"] in status}
266
267
268 def xjyl(problem, dzys=640):
269     return dict(
270         problem=problem,
271         extra=dzys,
272         coverage_nodes=20000,
273         coverage_depth=14,
274         channels={
275             c: dict(
276                 status="unknown",
277                 radio=[],
278                 optical=[],
279                 records=[],
280                 pending=set(g.sswd()),
281                 source=None,
282             )
283             for c in range(1, 21)
284         },
285     )
286
287
288 def xjlx(points, seconds=0.25):
289     return dict(
290         points=list(points),

```

```

291         seconds=seconds,
292         signature=None,
293         route=[],
294         pending_stop=None,
295     )
296
297
298 def xjzt(io, problem, jd=None, *, dzys=640, ghsj=0.2, qqsj=0.1):
299     return dict(
300         io=io,
301         problem=problem,
302         progress=jd,
303         ledger=xjzl(problem, dzys),
304         planner=dict(seconds=ghsj, problem=problem),
305         request_seconds=qqsj,
306         position=(0.0, 0.0),
307         channel=1,
308         virtual=0.0,
309         travel=0.0,
310         measures=0,
311         clear_attempts=0,
312         switches=0,
313         entered=False,
314         exit_confirmed=False,
315         exit_attempted=False,
316         completed=False,
317         deadline=math.inf,
318         max_virtual=360000.0,
319         fallback_used=False,
320         search_plan=xjlx(sscd(problem)),
321     )
322
323
324 def qjj(poly, other):
325     if len(other) < 3:
326         return poly
327     for a, b in zip(other, other[1:] + other[:1]):
328         edge = (b[0] - a[0], b[1] - a[1])
329         normal = (edge[1], -edge[0])
330         poly = g.clip(poly, normal, normal[0] * a[0] + normal[1] * a[1])
331         if not poly:
332             break
333     return poly
334
335
336 def xjyd(origin, bearing=None, problem=4):
337     origin = g.point(origin)
338     yuan = dict(
339         origin=origin,
340         bearing=bearing,
341         points={},
342         cells={},
343         polygon=[],
344         positives=[],
345         negatives=[],
346         anchor_point=origin,
347         anchor_bearing=bearing or 0.0,
348         anchor_radius=1500.0,
349         rounds=0,
350     )
351     domain = g.juxing(-1800, -1800, 1800, 1800)
352     if bearing is None:
353         x, y = origin
354         yuan["polygon"] = qjj(g.juxing(x - 5, y - 5, x + 5, y + 5), domain)
355         yuan["points"] = {0: origin}
356         yuan["cells"] = {0: yuan["polygon"]}
357         yuan["anchor_radius"] = 5.0
358     else:

```

```

359     yuan["polygon"] = g.jcfw(domain, origin, fs(bearing))
360     index = 0
361     for row, y in enumerate((-25, 0, 25)):
362         for k in range(61) if row % 2 == 0 else range(60, -1, -1):
363             x = 25 * k
364             bounds = (
365                 max(0, x - fs(25, 2)),
366                 max(-30, y - fs(25, 2)),
367                 min(1500, x + fs(25, 2)),
368                 min(30, y + fs(25, 2)),
369             )
370             cell = qjj(g.xzqj(origin, bearing, bounds), yuan["polygon"])
371             q = g.xzd(origin, bearing, x, y)
372             if cell:
373                 if not g.ypbh(cell, q, fs(20)):
374                     raise RuntimeError("Optical responsibility is not covered")
375                 yuan["points"][index], yuan["cells"][index] = (q, cell)
376                 index += 1
377     yuan["positives"].append(origin)
378     if not yuan["polygon"] or not yuan["cells"]:
379         raise RuntimeError("Known source lost all feasible responsibilities")
380     if problem == 3:
381         xdfw(yuan, qjj(yuan["polygon"], g.yfw()))
382         yuan["anchor_radius"] = math.nextafter(
383             max((float(g.jlsx(yuan["anchor_point"], p)) for p in yuan["polygon"])),
384             math.inf,
385         )
386     return yuan
387
388
389 def xdfw(yuan, poly):
390     if not poly:
391         raise RuntimeError("Accepted evidence contradicts known source")
392     yuan["cells"] = {i: out for i, p in yuan["cells"].items() if (out := qjj(p, poly))}
393     yuan["polygon"] = poly
394     if not yuan["polygon"] or not yuan["cells"]:
395         raise RuntimeError("Known source lost all feasible responsibilities")
396
397
398 def gxyd(yuan, q, kind, bearing=None, problem=4):
399     q = g.point(q)
400     if kind == "direction":
401         xdfw(yuan, g.jcfw(yuan["polygon"], q, fs(bearing)))
402         yuan["positives"].append(q)
403         yuan["anchor_point"], yuan["anchor_bearing"] = (q, bearing)
404         yuan["anchor_radius"] = min(
405             1500.0,
406             math.nextafter(
407                 float(max((g.jlsx(q, v) for v in yuan["polygon"]))), math.inf
408             ),
409         )
410         yuan["rounds"] += 1
411     elif kind == "near":
412         x, y = q
413         xdfw(yuan, qjj(yuan["polygon"], g.juxing(x - 5, y - 5, x + 5, y + 5)))
414         yuan["anchor_point"] = q
415         yuan["anchor_radius"] = math.nextafter(
416             float(max((g.jlsx(q, v) for v in yuan["polygon"]))), math.inf
417         )
418     elif kind == "no_signal":
419         yuan["negatives"].append(q)
420         if problem == 3:
421             poly = yuan["polygon"]
422             for p in yuan["positives"]:
423                 poly = g.clip(
424                     poly,
425                     (q[0] - p[0], q[1] - p[1]),
426                     (q[0] * q[0] + q[1] * q[1] - (p[0] * p[0] + p[1] * p[1])) / 2,

```

```

427         )
428         xdfw(yuan, poly)
429         yuan["cells"] = {
430             i: p
431             for i, p in yuan["cells"].items()
432             if problem != 3 or not g.ypbh(p, q, fs(1000))
433         }
434         if not yuan["polygon"] or not yuan["cells"]:
435             raise RuntimeError("Known source lost all feasible responsibilities")
436     elif kind == "no_target_in_range":
437         yuan["cells"] = {
438             i: p for i, p in yuan["cells"].items() if not g.ypbh(p, q, fs(20))
439         }
440         if not yuan["polygon"] or not yuan["cells"]:
441             raise RuntimeError("Known source lost all feasible responsibilities")
442         yuan["polygon"] = g.hull((v for p in yuan["cells"].values() for v in p))
443
444
445 def syx(yuan, pos, a, b):
446     if (
447         pos not in yuan["positives"]
448         or a not in yuan["negatives"]
449         or b not in yuan["negatives"]
450     ):
451         raise RuntimeError("Pair contraction lacks its observed evidence")
452     poly, changed = g.syx(yuan["polygon"], pos, a, b)
453     if changed:
454         xdfw(yuan, poly)
455         yuan["anchor_radius"] = min(
456             yuan["anchor_radius"],
457             math.nextafter(
458                 float(max((g.jlsx(yuan["anchor_point"], v) for v in poly))), math.inf
459             ),
460         )
461     yuan["rounds"] += 1
462     return changed
463
464
465 def syrw(jilu):
466     return (
467         sum(
468             (
469                 len(s["pending"])
470                 for s in jilu["channels"].values()
471                 if s["status"] == "unknown"
472             )
473         )
474         + sum(
475             (
476                 len(s["source"]["cells"])
477                 for s in jilu["channels"].values()
478                 if s["status"] == "found"
479             )
480         )
481         + 183 * (16 - len(pindao(jilu, ("found", "cleared"))))
482         + jilu["extra"]
483     )
484
485
486 def bjwy(jilu, c):
487     s = jilu["channels"][c]
488     s["status"] = "absent"
489     s["pending"].clear()
490
491
492 def pdwy(jilu, c):
493     s = jilu["channels"][c]
494     if s["status"] != "unknown":

```

```

495         return False
496     if not s["pending"]:
497         bjwy(jilu, c)
498         return True
499     if len(s["radio"]) < (6 if jilu["problem"] == 3 else 7) and (not s["optical"]):
500         return False
501     covered = g.fgpd(
502         tuple(sorted(set(s["radio"]))),
503         tuple(sorted(set(s["optical"]))),
504         jilu["problem"],
505         jilu["coverage_nodes"],
506         jilu["coverage_depth"],
507     )
508     if covered:
509         bjwy(jilu, c)
510     return covered
511
512
513 def gxjl(jilu, path, q, c, huifu, pd=True):
514     before = syrjw(jilu)
515     q = g.point(q)
516     s = jilu["channels"][c]
517     if s["status"] in ("cleared", "absent"):
518         raise RuntimeError("Action targets a resolved channel")
519     kind = huifu["measure_result"] if path == "/measure" else "clear_result"
520     s["records"].append(
521         dict(
522             path=path,
523             point=tuple((float(x) for x in q)),
524             kind=kind,
525             bearing=huifu.get("svd_deg"),
526         )
527     )
528     if path == "/measure":
529         if s["status"] == "unknown":
530             if kind == "no_signal":
531                 s["radio"].append(q)
532                 s["pending"].discard(q)
533                 if pd:
534                     pdwy(jilu, c)
535             else:
536                 s["status"] = "found"
537                 s["pending"].clear()
538                 s["source"] = xjyd(
539                     q,
540                     huifu["svd_deg"] if kind == "direction" else None,
541                     jilu["problem"],
542                 )
543             elif kind == "direction":
544                 gxyd(s["source"], q, "direction", bearing=huifu["svd_deg"])
545             elif kind == "near":
546                 gxyd(s["source"], q, "near")
547             else:
548                 gxyd(s["source"], q, "no_signal", problem=jilu["problem"])
549         elif kind == "success":
550             s["status"] = "cleared"
551             s["pending"].clear()
552             s["source"] = None
553         elif s["status"] == "unknown":
554             s["optical"].append(q)
555             if pd:
556                 pdwy(jilu, c)
557         else:
558             gxyd(s["source"], q, "no_target_in_range")
559     if len(pindao(jilu, ("found", "cleared"))) > 16:
560         raise RuntimeError("More than 16 distinct sources")
561     if len(pindao(jilu, ("found", "cleared"))) == 16:
562         for other in sorted(pindao(jilu, ("unknown",))):

```

```

563         bjwy(jilu, other)
564     if syrw(jilu) >= before:
565         if jilu["extra"] <= 0:
566             raise RuntimeError("An unproductive action had no reserved allowance")
567         jilu["extra"] -= 1
568
569
570 def qbqc(jilu):
571     return (
572         not pindao(jilu, ("unknown",))
573         and (not pindao(jilu, ("found",)))
574         and (10 <= len(pindao(jilu, ("cleared",))) <= 16)
575     )
576
577
578 def sysj(jilu, weizhi):
579     p = g.point(weizhi)
580     parts = [
581         lxsj(gxpx(jilu["channels"][c]["source"], p), p, p)
582         for c in pindao(jilu, ("found",))
583     ]
584     points = set(
585         (q for c in pindao(jilu, ("unknown",)) for q in jilu["channels"][c]["pending"])
586     )
587     prev = p
588     for q in g.sswd():
589         if q in points:
590             parts.append(math.nextafter(float(g.jlsx(prev, q) / 5), math.inf))
591             prev = q
592     requests = sum(
593         (len(jilu["channels"][c]["pending"]) for c in pindao(jilu, ("unknown",)))
594     )
595     parts.append(6 * requests)
596     parts.append(
597         math.nextafter(
598             1843.1
599             * min(
600                 16 - len(pindao(jilu, ("found", "cleared"))),
601                 len(pindao(jilu, ("unknown",))),
602             ),
603             math.inf,
604         )
605     )
606     return math.nextafter(math.fsum(parts), math.inf)
607
608
609 def lxqc(yuan, dq, target=None):
610     polygon = [tuple((float(x) for x in p)) for p in yuan["polygon"]]
611     cand = zjqcd(polygon, dq)
612     if cand is None:
613         return None
614     options = [cand]
615     if target is not None:
616         q, _ = qclxyh(polygon, 19.8 - 1e-06, cand, dq, target)
617         options.append(q)
618     valid = [
619         g.point(q) for q in options if g.ypbh(yuan["polygon"], g.point(q), fs("19.8"))
620     ]
621     if not valid:
622         return None
623     return min(
624         valid,
625         key=lambda q: (
626             math.dist(dq, tuple((float(x) for x in q)))
627             + (
628                 math.dist(tuple((float(x) for x in q)), target)
629                 if target is not None
630                 else 0

```

```

631         )
632     ),
633 )
634
635
636 def gxpx(yuan, dq, mode="snake", cells=None):
637     ids = set(yuan["cells"] if cells is None else cells)
638     if mode == "snake":
639         return [yuan["points"][i] for i in sorted(ids)]
640     order = []
641     p = g.point(dq)
642     while ids:
643         i = min(
644             ids,
645             key=lambda i: (
646                 (p[0] - yuan["points"][i][0]) * (p[0] - yuan["points"][i][0])
647                 + (p[1] - yuan["points"][i][1]) * (p[1] - yuan["points"][i][1]),
648                 i,
649             ),
650         )
651         ids.remove(i)
652         p = yuan["points"][i]
653         order.append(p)
654     return order
655
656
657 def lxsj(points, dq, target=None):
658     p = g.point(dq)
659     spent = fs(0)
660     worst = fs(0)
661     for q in points:
662         spent += g.jlsx(p, q) / 5
663         leave = g.jlsx(q, g.point(target)) / 5 if target is not None else 0
664         worst = max(worst, spent + 5 + leave)
665         spent += 3
666         p = q
667     return math.nextafter(float(worst), math.inf)
668
669
670 def xuduan(yuan, dq, cells=None):
671     return min(
672         (
673             (lxsj(order, dq), mode, order)
674             for mode in ("snake", "nearest")
675             if (order := gxpx(yuan, dq, mode, cells))
676         )
677     )
678
679
680 def sdc(yuan, dq, ratio=0.5):
681     length = fs(yuan["anchor_radius"])
682     x = 5 + (length - 5) * fs(ratio)
683     side = length / 50
684     if len(yuan["positives"]) >= 2:
685         u, _ = g.fangxiang(fs(yuan["anchor_bearing"]))
686         dist = [
687             (q[0] - yuan["anchor_point"][0]) * u[0]
688             + (q[1] - yuan["anchor_point"][1]) * u[1]
689             for q in yuan["polygon"]
690         ]
691         lo, hi = (min(dist), max(dist))
692         x = (lo + hi) / 2
693         side = max(fs(5), (hi - lo) / 20)
694     points = [
695         g.xzd(yuan["anchor_point"], yuan["anchor_bearing"], x, sign * side)
696         for sign in (-1, 1)
697     ]
698     return tuple(

```

```

699         sorted(
700             points,
701             key=lambda q: (
702                 (q[0] - g.point(dq)[0]) * (q[0] - g.point(dq)[0])
703                 + (q[1] - g.point(dq)[1]) * (q[1] - g.point(dq)[1])
704             ),
705         )
706     )
707
708
709 def kdqy(yuan, polygon):
710     if not polygon:
711         return []
712     return [i for i, cell in yuan["cells"].items() if qjj(cell, polygon)]
713
714
715 def sptz(yuan, dq, pair, problem, dl):
716     a, b = pair
717     start = float(g.jlsx(g.point(dq), a) / 5) + 6
718     ceshi = [(a, start)]
719     if problem == 4:
720         ceshi.append((b, start + float(g.jlsx(a, b) / 5) + 5))
721     worst = 0.0
722     paths = 0
723     for q, prefix in ceshi:
724         for lo in range(0, 360, 60):
725             if time.perf_counter() >= dl:
726                 return None
727             region = g.jcfw(yuan["polygon"], q, fs(lo), hi=fs(lo + 60))
728             ids = kdqy(yuan, region)
729             if ids:
730                 upper, _, _ = xuduan(yuan, q, ids)
731                 worst = max(worst, prefix + upper)
732                 paths += 1
733             worst = max(worst, prefix + 5)
734             paths += 1
735     if problem == 3:
736         if not g.ypbh(yuan["polygon"], a, fs(1000)):
737             worst = max(worst, start + xuduan(yuan, a)[0])
738             paths += 1
739     else:
740         prefix = ceshi[-1][1]
741         region, _ = g.syxc(yuan["polygon"], yuan["anchor_point"], a, b)
742         ids = kdqy(yuan, region)
743         if ids:
744             worst = max(worst, prefix + xuduan(yuan, b, ids)[0])
745             paths += 1
746     return (worst, paths)
747
748
749 def gxjs(source):
750     values = []
751     for poly in source["cells"].values():
752         points = [tuple((float(x) for x in p)) for p in poly]
753         center = tuple((sum((p[k] for p in points)) / len(points) for k in (0, 1)))
754         area = (
755             abs(
756                 sum(
757                     (
758                         a[0] * b[1] - a[1] * b[0]
759                         for a, b in zip(points, points[1:] + points[:1])
760                     )
761                 )
762             )
763             / 2
764         )
765         values.append((center, area))
766     z1 = sum((w for _, w in values))

```

```

767     return [(p, w / z1 if z1 > 1e-12 else 1 / len(values)) for p, w in values]
768
769
770 def gxjz(order, wz, worlds, target=None):
771     elapsed = 0.0
772     prev = wz
773     rest = set(range(len(worlds)))
774     mean = 0.0
775     xout = yout = 0.0
776     for q in order:
777         q = tuple((float(x) for x in q))
778         elapsed += math.dist(prev, q) / 5
779         hits = [i for i in rest if math.dist(worlds[i][0], q) <= 20]
780         for i in hits:
781             mean += worlds[i][1] * (
782                 elapsed + 5 + (math.dist(q, target) / 5 if target is not None else 0)
783             )
784             xout += worlds[i][1] * q[0]
785             yout += worlds[i][1] * q[1]
786             rest.remove(i)
787         prev = q
788         elapsed += 3
789     return (math.inf if rest else mean, (xout, yout))
790
791
792 def gxbx(source, wz, target=None):
793     poly = source["polygon"]
794     points = [tuple((float(x) for x in p)) for p in poly]
795     angle = math.radians(source["anchor_bearing"])
796     result = []
797     worlds = None
798     for theta in (angle, angle + math.pi / 2):
799         u = (math.cos(theta), math.sin(theta))
800         w = (-u[1], u[0])
801         xs = [p[0] * u[0] + p[1] * u[1] for p in points]
802         ys = [p[0] * w[0] + p[1] * w[1] for p in points]
803         lo, hi = (min(xs) - 1e-06, max(xs) + 1e-06)
804         mid = (min(ys) + max(ys)) / 2
805         halfwidth = (max(ys) - min(ys)) / 2 + 1e-06
806         if halfwidth >= 19.79:
807             continue
808         half = math.sqrt(19.79**2 - halfwidth**2)
809         n = max(1, math.ceil((hi - lo) / (2 * half)))
810         if not 2 <= n <= 8:
811             continue
812         centers = [lo + half + (hi - lo - 2 * half) * i / (n - 1) for i in range(n)]
813         chain = [
814             g.point((x * u[0] + mid * w[0], x * u[1] + mid * w[1])) for x in centers
815         ]
816         valid = True
817         for q in chain:
818             part = poly
819             for other in chain:
820                 if q != other:
821                     part = g.clip(
822                         part,
823                         (other[0] - q[0], other[1] - q[1]),
824                         (
825                             other[0] * other[0]
826                             + other[1] * other[1]
827                             - (q[0] * q[0] + q[1] * q[1])
828                         )
829                         / 2,
830                     )
831             if part and (not g.ypbh(part, q, fs("19.8"))):
832                 valid = False
833                 break
834         if not valid:

```

```

835         continue
836     if worlds is None:
837         worlds = gxjs(source)
838     greedy = []
839     left = list(chain)
840     now = wz
841     rest = set(range(len(worlds)))
842     while left:
843         q = min(
844             left,
845             key=lambda p: (
846                 -sum(
847                     (
848                         worlds[i][1]
849                         for i in rest
850                         if math.dist(worlds[i][0], tuple((float(x) for x in p)))
851                         <= 20
852                     )
853                 )
854                 / (3 + math.dist(now, tuple((float(x) for x in p))) / 5),
855                 (g.point(now)[0] - p[0]) * (g.point(now)[0] - p[0])
856                 + (g.point(now)[1] - p[1]) * (g.point(now)[1] - p[1]),
857             ),
858         )
859         greedy.append(q)
860         left.remove(q)
861         now = tuple((float(x) for x in q))
862         rest = {i for i in rest if math.dist(worlds[i][0], now) > 20}
863     for order in dict.fromkeys(
864         (tuple(chain), tuple(reversed(chain)), tuple(greedy))
865     ):
866         mean, leave = gxjz(order, wz, worlds, target)
867         if math.isfinite(mean):
868             result.append((mean, order, leave, lxsj(order, wz, target)))
869     return result
870
871
872 def gxxz(jihua, yuan, channel, wz, tuned, extra, event, target):
873     if jihua["problem"] != 4 or event["method"] == "verified_continuous" or extra < 8:
874         return event
875     options = gxbx(yuan, wz, target)
876     if not options:
877         return event
878     worlds = gxjs(yuan)
879     q = tuple((float(x) for x in event["points"][0]))
880     ideal = math.dist(wz, q) / 5 + 5 + (channel != tuned) + 5
881     ideal += sum(
882         (
883             weight
884             * (
885                 max(0, math.dist(q, source) - 19.8) / 5
886                 + (
887                     max(0, math.dist(source, target) - 19.8) / 5
888                     if target is not None
889                     else 0
890                 )
891             )
892             for source, weight in worlds
893         )
894     )
895     mean, order, leave, upper = min(options, key=lambda option: option[0])
896     if mean < ideal:
897         return dict(
898             kind="optical_chain",
899             channel=channel,
900             points=order,
901             estimate=mean,
902             method="complete_cover_expected_first_success",

```

```

903         )
904     return event
905
906
907 def yddz(jihua, yuan, channel, dq, tuned, extra, target=None):
908     q = lxqc(yuan, dq, target)
909     if q is not None:
910         return {
911             "kind": "clear",
912             "channel": channel,
913             "points": (q,),
914             "estimate": math.dist(dq, tuple((float(x) for x in q))) / 5 + 5,
915             "method": "verified_continuous",
916         }
917     upper, mode, order = xuduan(yuan, dq)
918     best = {
919         "kind": "clear",
920         "channel": channel,
921         "points": (order[0],),
922         "estimate": upper,
923         "method": "complete_optical_" + mode,
924     }
925     if extra < 2 or yuan["anchor_radius"] <= 5 or yuan["rounds"] >= 12:
926         return gxxz(jihua, yuan, channel, dq, tuned, extra, best, target)
927     dl = time.perf_counter() + jihua["seconds"]
928     for ratio in (0.5, 0.38, 0.62):
929         pair = sdc(yuan, dq, ratio)
930         jieguo = sptz(yuan, dq, pair, jihua["problem"], dl)
931         if jieguo is None:
932             break
933         value, paths = jieguo
934         if value < best["estimate"]:
935             best = {
936                 "kind": "radio",
937                 "channel": channel,
938                 "points": pair,
939                 "estimate": value,
940                 "method": f"interval_rollout_{ratio:g}_{paths}_branches",
941             }
942     if best["kind"] == "clear" and len(yuan["cells"]) > 4:
943         pair = sdc(yuan, dq)
944         best = {
945             "kind": "radio",
946             "channel": channel,
947             "points": pair,
948             "estimate": best["estimate"],
949             "method": "annular_proposal_with_guard",
950         }
951     best = bmcf(jihua, yuan, dq, best)
952     best = gxxz(jihua, yuan, channel, dq, tuned, extra, best, target)
953     return best
954
955
956 def bmcf(jihua, yuan, wz, event):
957     if jihua["problem"] != 4 or event["kind"] != "radio":
958         return event
959     if not all((q in yuan["negatives"] for q in event["points"])):
960         return event
961     upper, _, order = xuduan(yuan, wz)
962     return dict(
963         event,
964         kind="clear",
965         points=(order[0],),
966         estimate=upper,
967         method="avoid_repeated_negative_pair",
968     )
969
970

```

```

971 def pgsx(tu, order):
972     if not order:
973         return (0.0, [])
974     costs = {i: (tu["start"][i], [i]) for i in tu["ids"][order[0]]}
975     for job in order[1:]:
976         costs = {
977             j: min(
978                 (
979                     (value + tu["edge"][i, j], path + [j])
980                     for i, (value, path) in costs.items()
981                 )
982             )
983             for j in tu["ids"][job]
984         }
985     value, path = min(costs.values())
986     return (value, [tu["modes"][i] for i in path])
987
988
989 def fpsx(tu):
990     jobs = list(tu["ids"])
991     n = len(jobs)
992     cost = [[1000000000000.0] * (n + 1) for _ in range(n + 1)]
993     for j, job in enumerate(jobs, 1):
994         cost[0][j] = min((tu["start"][k] for k in tu["ids"][job]))
995     for i, a in enumerate(jobs, 1):
996         cost[i][0] = 0.0
997         for j, b in enumerate(jobs, 1):
998             if i != j:
999                 cost[i][j] = min(
1000                     (tu["edge"][x, y] for x in tu["ids"][a] for y in tu["ids"][b])
1001                 )
1002     _, succ = zhipai(cost)
1003
1004     def cycle(start):
1005         out = [start]
1006         q = succ[start]
1007         while q != start:
1008             out.append(q)
1009             q = succ[q]
1010         return out
1011
1012     main = cycle(0)
1013     while len(main) < n + 1:
1014         other = cycle(next((i for i in range(n + 1) if i not in main)))
1015         a, b = min(
1016             ((a, b) for a in main for b in other),
1017             key=lambda ab: (
1018                 cost[ab[0]][succ[ab[1]]]
1019                 + cost[ab[1]][succ[ab[0]]]
1020                 - cost[ab[0]][succ[ab[0]]]
1021                 - cost[ab[1]][succ[ab[1]]]
1022             ),
1023         )
1024         succ[a], succ[b] = (succ[b], succ[a])
1025         main = cycle(0)
1026     return [jobs[i - 1] for i in main[1:]]
1027
1028
1029 def aprw(jobs, wz, oldorder=()):
1030     modes = [m for opts in jobs.values() for m in opts]
1031     tu = dict(
1032         modes=modes,
1033         ids={job: [i for i, m in enumerate(modes) if m["job"] == job] for job in jobs},
1034         edge={
1035             (i, j): math.dist(a["exit"], b["entry"]) / 5 + b["service"]
1036             for i, a in enumerate(modes)
1037             for j, b in enumerate(modes)
1038             if a["job"] != b["job"]

```

```

1039     },
1040     start={
1041         i: math.dist(wz, m["entry"]) / 5 + m["service"] for i, m in enumerate(modes)
1042     },
1043 )
1044 old = [job for job in oldorder if job in jobs]
1045 old += [job for job in jobs if job not in old]
1046 rest = list(jobs)
1047 greedy = []
1048 last = None
1049 while rest:
1050     job = min(
1051         rest,
1052         key=lambda job: min(
1053             (
1054                 tu["start"][i] if last is None else tu["edge"][last, i]
1055                 for i in tu["ids"][job]
1056             )
1057         ),
1058     )
1059     last = min(
1060         tu["ids"][job],
1061         key=lambda i: tu["start"][i] if last is None else tu["edge"][last, i],
1062     )
1063     greedy.append(job)
1064     rest.remove(job)
1065 scans = [job for job in jobs if job[0] == "scan"]
1066 sources = [job for job in jobs if job[0] == "source"]
1067 seeds = [old, greedy, sources + scans, scans + sources, fpsx(tu)]
1068 scores = []
1069 for seed in seeds:
1070     value, opts = pgxs(tu, seed)
1071     scores.append((value, seed, opts))
1072 value, order, opts = min(scores, key=lambda e: e[0])
1073 index = {id(m): i for i, m in enumerate(modes)}
1074 for _ in range(2):
1075     before = value
1076     trials = []
1077     chosen = {m["job"]: index[id(m)] for m in opts}
1078     for i in range(len(order)):
1079         for j in range(len(order)):
1080             if i == j:
1081                 continue
1082             trial = list(order)
1083             item = trial.pop(i)
1084             trial.insert(j, item)
1085             seq = [chosen[job] for job in trial]
1086             score = tu["start"][seq[0]] + sum(
1087                 (tu["edge"][a, b] for a, b in zip(seq, seq[1:]))
1088             )
1089             if score < value - 1e-07:
1090                 trials.append((score, trial))
1091     for _, trial in sorted(trials, key=lambda x: x[0])[:16]:
1092         cand, plan = pgxs(tu, trial)
1093         if cand < value - 1e-07:
1094             value, order, opts = (cand, trial, plan)
1095     if value >= before - 1e-07:
1096         break
1097     return dict(estimate=value, order=order, modes=opts)
1098
1099
1100 def xzdz(jihua, jilu, dq, tuned, sscd):
1101     jobs = {}
1102     for c in sorted(pindao(jilu, ("found",))):
1103         yuan = jilu["channels"][c]["source"]
1104         job = ("source", c)
1105         direct = lxqc(yuan, dq)
1106         if direct is not None:

```

```

1107         q = tuple((float(x) for x in direct))
1108         event = dict(
1109             kind="clear",
1110             channel=c,
1111             points=(direct,),
1112             estimate=5.0,
1113             method="service_direct_clear",
1114         )
1115         modes = [dict(job=job, entry=q, service=5.0, exit=q, event=event)]
1116     else:
1117         center = tuple(
1118             (
1119                 sum((float(q[k]) for q in yuan["polygon"])) / len(yuan["polygon"])
1120                 for k in (0, 1)
1121             )
1122         )
1123         modes = []
1124         for ratio in (0.38, 0.5, 0.62):
1125             pair = sdc(yuan, dq, ratio)
1126             for order in (pair, pair[::-1]) if jilu["problem"] == 4 else (pair,):
1127                 q = tuple((float(x) for x in order[0]))
1128                 cost = math.dist(q, center) / 5 + 15.0
1129                 event = dict(
1130                     kind="radio",
1131                     channel=c,
1132                     points=order,
1133                     estimate=cost,
1134                     method="service_entry_exit",
1135                 )
1136                 modes.append(
1137                     dict(job=job, entry=q, service=cost, exit=center, event=event)
1138                 )
1139         jobs[job] = modes
1140     for q in sscd:
1141         todo = [
1142             c
1143             for c in pindao(jilu, ("unknown",))
1144             if g.point(q) not in jilu["channels"][c]["radio"]
1145         ]
1146         if todo:
1147             job = ("scan", tuple(q))
1148             event = dict(
1149                 kind="search",
1150                 channel=min(todo, key=lambda c: (c != tuned, c)),
1151                 points=(g.point(q),),
1152                 estimate=6 * len(todo),
1153                 method="service_search",
1154             )
1155             jobs[job] = [
1156                 dict(job=job, entry=q, service=event["estimate"], exit=q, event=event)
1157             ]
1158         if not jobs:
1159             return None
1160         plan = aprw(jobs, dq, jihua.get("order", ()))
1161         jihua["order"] = plan["order"]
1162         event = plan["modes"][0]["event"]
1163         event["target"] = plan["modes"][1]["entry"] if len(plan["modes"]) > 1 else None
1164         if event["kind"] != "search":
1165             event = bmcf(jihua, jilu["channels"][event["channel"]]["source"], dq, event)
1166         return event
1167
1168
1169 def gxlx(luxian, jilu, dq):
1170     if not pindao(jilu, ("unknown",)):
1171         return []
1172     common = set.intersection(
1173         *(set(jilu["channels"][c]["radio"]) for c in pindao(jilu, ("unknown",)))
1174     )

```

```

1175     sycd = [q for q in luxian["points"] if g.point(q) not in common]
1176     sig = (
1177         tuple(sorted(common)),
1178         tuple(sorted(pindao(jilu, ("unknown",)))),
1179         luxian["pending_stop"],
1180     )
1181     route = lxxp(dq, sycd)
1182     if sig == luxian["signature"]:
1183         return route
1184     luxian["signature"] = sig
1185     dl = time.perf_counter() + luxian["seconds"]
1186
1187     def cost(qs):
1188         return lxxd(dq, qs) / 5 + 6 * len(pindao(jilu, ("unknown",))) * len(qs)
1189
1190     for _ in range(3):
1191         old = cost(route)
1192         options = []
1193         for q in route:
1194             rest = [p for p in route if p != q]
1195             trial = lxxp(dq, rest)
1196             options.append((cost(trial), "delete", q, trial))
1197             if (
1198                 luxian["pending_stop"] is not None
1199                 and luxian["pending_stop"] not in rest
1200                 and (g.point(luxian["pending_stop"]) not in common)
1201             ):
1202                 trial = lxxp(dq, rest + [luxian["pending_stop"]])
1203                 options.append((cost(trial), "replace", q, trial))
1204             improved = False
1205             for value, kind, removed, trial in sorted(options):
1206                 if value >= old - 1e-06 or time.perf_counter() >= dl:
1207                     break
1208                 points = tuple(sorted(common | {g.point(q) for q in trial}))
1209                 covered = g.fgpd(
1210                     points,
1211                     (),
1212                     jilu["problem"],
1213                     jilu["coverage_nodes"],
1214                     jilu["coverage_depth"],
1215                 )
1216                 if covered:
1217                     route = trial
1218                     improved = True
1219                     break
1220             if not improved:
1221                 break
1222     luxian["points"] = route
1223     luxian["route"] = route
1224     luxian["pending_stop"] = None
1225     luxian["signature"] = (
1226         tuple(sorted(common)),
1227         tuple(sorted(pindao(jilu, ("unknown",)))),
1228         None,
1229     )
1230     return route
1231
1232
1233 def bcsj(zt, event, **fields):
1234     data = dict(event=event, virtual_time_s=zt["virtual"], **fields)
1235     zt["io"]["record"](data)
1236     if zt["progress"]:
1237         zt["progress"](data)
1238
1239
1240 def syics(zt):
1241     l = zt["ledger"]
1242     return (

```

```

1243         sum((len(l["channels"][c]["pending"]) for c in pindao(l, ("unknown",))))
1244         + sum((len(l["channels"][c]["source"]["cells"]) for c in pindao(l, ("found",))))
1245         + 183
1246         * min(16 - len(pindao(l, ("found", "cleared"))), len(pindao(l, ("unknown",))))
1247         + 2
1248     )
1249
1250
1251 def ysjc(zt, points):
1252     jilu = zt["ledger"]
1253     upper = sysj(jilu, zt["position"])
1254     p = zt["position"]
1255     dist = 0.0
1256     for q in points:
1257         dist += float(g.jlsx(g.point(p), g.point(q)))
1258         p = tuple((float(x) for x in q))
1259     m = min(16, len(pindao(jilu, ("found",))) + len(points))
1260     virtualok = (
1261         zt["virtual"] + upper + (2 * m + 2) * dist / 5 + 6 * len(points)
1262         < zt["max_virtual"] - 2
1263     )
1264     realok = (
1265         time.monotonic() + (syscs(zt) + len(points)) * zt["request_seconds"] + 30
1266         < zt["deadline"]
1267     )
1268     return virtualok and realok
1269
1270
1271 def zhixing(zt, path, q, c, *, pd=True):
1272     jiekou = zt["io"]
1273     jilu = zt["ledger"]
1274     q = tuple((float(x) for x in g.point(q)))
1275     if jiekou["pending"] is not None:
1276         raise RuntimeError("Earlier request unresolved")
1277     if time.monotonic() >= zt["deadline"] - 2:
1278         raise TimeoutError("Real deadline reserve reached without completion")
1279     moved = math.dist(zt["position"], q)
1280     if zt["virtual"] + moved / 5 + 6 >= zt["max_virtual"] - 1:
1281         raise RuntimeError("Physical action exceeds virtual budget")
1282     sw = int(path == "/measure" and c != zt["channel"])
1283     huifu = jiekou["call"](path, q, c)
1284     zt["position"] = q
1285     zt["virtual"] = huifu["virtual_time_s"]
1286     zt["travel"] += moved
1287     if path == "/measure":
1288         zt["measures"] += 1
1289         zt["switches"] += sw
1290         zt["channel"] = c
1291     else:
1292         zt["clear_attempts"] += 1
1293         found = c in pindao(jilu, ("found", "cleared"))
1294         gxjl(jilu, path, q, c, huifu, pd=pd)
1295         if not found and c in pindao(jilu, ("found", "cleared")):
1296             bcsj(zt, "discovered", channel=c)
1297         if path == "/clear" and huifu["clear_result"] == "success":
1298             zt["search_plan"]["pending_stop"] = tuple(q)
1299             bcsj(zt, "cleared", channel=c, cleared_count=len(pindao(jilu, ("cleared",))))
1300     return huifu
1301
1302
1303 def fxjd(p, q):
1304     if p[0] == q[0] and p[1] == q[1]:
1305         return [(0.0, 360.0)]
1306     angle = math.degrees(math.atan2(q[1] - p[1], q[0] - p[0]))
1307     lo = (angle - 90) % 360
1308     return [(lo, min(360, lo + 180))] + ([0, lo - 180] if lo > 180 else [])
1309
1310

```

```

1311 def jdjj(a, b):
1312     return [(max(x, u), min(y, w)) for x, y in a for u, w in b if max(x, u) < min(y, w)]
1313
1314
1315 def jdcj(a, b):
1316     for lo, hi in b:
1317         a = [
1318             piece
1319             for x, y in a
1320             for piece in ((x, min(y, lo)), (max(x, hi), y))
1321             if piece[0] < piece[1]
1322         ]
1323     return a
1324
1325
1326 def jdcd(spans):
1327     return sum((y - x for x, y in spans))
1328
1329
1330 def spmx(record):
1331     src = record["source"]
1332     pos = []
1333     neg = []
1334     for obs in record["records"]:
1335         if obs["path"] != "/measure":
1336             continue
1337         (neg if obs["kind"] == "no_signal" else pos).append(obs["point"])
1338     zl = 0.0
1339     jiashuo = []
1340     for poly in src["cells"].values():
1341         points = [tuple((float(x) for x in q)) for q in poly]
1342         p = tuple((sum((q[k] for q in points)) / len(points) for k in (0, 1)))
1343         area = (
1344             abs(
1345                 sum(
1346                     (
1347                         a[0] * b[1] - a[1] * b[0]
1348                         for a, b in zip(points, points[1:] + points[:1])
1349                     )
1350                 )
1351             )
1352             / 2
1353         )
1354         lower = max([1000] + [math.dist(p, q) for q in pos])
1355         upper = 1500.0
1356         if lower >= upper or area <= 1e-12:
1357             continue
1358         valid = [(0.0, 360.0)]
1359         for q in pos:
1360             valid = jdjj(valid, fxjd(p, q))
1361         negs = [(math.dist(p, q), q) for q in neg]
1362         radii = sorted([lower, upper] + [d for d, q in negs if lower < d < upper])
1363         for lo, hi in zip(radii, radii[1:]):
1364             allowed = valid
1365             omni = True
1366             for d, q in negs:
1367                 if d < (lo + hi) / 2:
1368                     allowed = jdcj(allowed, fxjd(p, q))
1369             omni = False
1370             mass = area * (hi - lo) * (0.5 * omni + 0.5 * jdcd(allowed) / 360)
1371             if mass > 0:
1372                 zl += mass
1373             jiashuo.append((p, lo, hi, allowed, omni, area))
1374     return (jiashuo, zl)
1375
1376
1377 def spgl(fitted, q):
1378     jiashuo, zl = fitted

```

```

1379     if z1 <= 0:
1380         return 0.5
1381     kejian = 0.0
1382     for p, lo, hi, allowed, omni, area in jiashuo:
1383         width = max(0, hi - max(lo, math.dist(p, q)))
1384         if width:
1385             kejian += (
1386                 area
1387                 * width
1388                 * (0.5 * omni + 0.5 * jdcd(jdjj(allowed, fxjd(p, q))) / 360)
1389             )
1390     return min(1, max(0, kejian / z1))
1391
1392
1393 def slcl(z):
1394     jilu = z["ledger"]
1395     wz = z["position"]
1396     beixuan = []
1397     for c in sorted(pindao(jilu, ("found",))):
1398         yuan = jilu["channels"][c]["source"]
1399         tries = len(yuan["positives"]) + (
1400             len(yuan["negatives"]) if jilu["problem"] == 3 else 0
1401         )
1402         if tries >= 3 or g.point(wz) in yuan["positives"] + yuan["negatives"]:
1403             continue
1404         poly = [tuple((float(x) for x in p)) for p in yuan["polygon"]]
1405         center = tuple((sum((p[k] for p in poly)) / len(poly) for k in (0, 1)))
1406         if jilu["problem"] == 3:
1407             if math.dist(wz, center) > 1000:
1408                 continue
1409         elif not g.ypbh(yuan["polygon"], g.point(wz), fs(999)):
1410             continue
1411         angle = math.radians(yuan["anchor_bearing"])
1412         u = (math.cos(angle), math.sin(angle))
1413         origin = tuple((float(x) for x in yuan["anchor_point"]))
1414         span = [(p[0] - origin[0]) * u[0] + (p[1] - origin[1]) * u[1] for p in poly]
1415         width = max(span) - min(span)
1416         ray = (center[0] - wz[0], center[1] - wz[1])
1417         dist = math.hypot(*ray)
1418         parallax = abs(u[0] * ray[1] - u[1] * ray[0]) / max(1.0, dist)
1419         after = min(width, 2 * math.radians(1.005) * dist / max(0.01, parallax))
1420         recv = spgl(spmx(jilu["channels"][c]), wz) if jilu["problem"] == 4 else 1
1421         gain = recv * (width - after) / 10 - 6
1422         if parallax >= 0.25 and gain > 0:
1423             beixuan.append((-gain, c))
1424     for score, c in sorted(beixuan):
1425         if jilu["extra"] < 2 or not ysjc(z, (wz,)):
1426             return False
1427         zhixing(z, "/measure", wz, c)
1428     return True
1429
1430
1431 def plcl(z, unknown):
1432     jilu = z["ledger"]
1433     pd = [
1434         c
1435         for c in sorted(unknown & pindao(jilu, ("found",)))
1436         if len(jilu["channels"][c]["source"]["positives"]) == 1
1437     ]
1438     if len(pd) < 3 or jilu["extra"] < len(pd) + 2:
1439         return True
1440     wz = z["position"]
1441     sources = []
1442     for c in pd:
1443         yuan = jilu["channels"][c]["source"]
1444         poly = [tuple((float(x) for x in p)) for p in yuan["polygon"]]
1445         center = tuple((sum((p[k] for p in poly)) / len(poly) for k in (0, 1)))
1446         angle = math.radians(yuan["anchor_bearing"])

```

```

1447     u = (math.cos(angle), math.sin(angle))
1448     origin = tuple((float(x) for x in yuan["anchor_point"]))
1449     span = [(p[0] - origin[0]) * u[0] + (p[1] - origin[1]) * u[1] for p in poly]
1450     sources.append((center, u, max(span) - min(span)))
1451 targets = [
1452     tuple(
1453         (
1454             sum((float(p[k]) for p in jilu["channels"][c]["source"]["polygon"]))
1455             / len(jilu["channels"][c]["source"]["polygon"])
1456             for k in (0, 1)
1457         )
1458     )
1459     for c in pindao(jilu, ("found",))
1460 ]
1461 target = min(targets, key=lambda q: math.dist(wz, q))
1462 fitted = (
1463     [spmj(jilu["channels"][c]) for c in pd]
1464     if jilu["problem"] == 4
1465     else [None] * len(pd)
1466 )
1467 radius = 350
1468 options = []
1469 for angle in range(0, 360, 30):
1470     q = (
1471         wz[0] + radius * math.cos(math.radians(angle)),
1472         wz[1] + radius * math.sin(math.radians(angle)),
1473     )
1474     gain = 0.0
1475     chosen = []
1476     for c, fit, (center, u, width) in zip(pd, fitted, sources):
1477         ray = (center[0] - q[0], center[1] - q[1])
1478         dist = math.hypot(*ray)
1479         parallax = abs(u[0] * ray[1] - u[1] * ray[0]) / max(1.0, dist)
1480         postwidth = min(width, 2 * math.radians(1.005) * dist / max(0.01, parallax))
1481         benefit = (width - postwidth) / 10.0
1482         if jilu["problem"] == 4:
1483             benefit = spgl(fit, q) * benefit - 6
1484         if jilu["problem"] == 3 or benefit > 0:
1485             gain += benefit
1486             chosen.append(c)
1487         detour = (radius + math.dist(q, target) - math.dist(wz, target)) / 5.0
1488         options.append((detour - gain, angle, q, gain, detour, chosen))
1489     score, angle, q, gain, detour, pd = min(options)
1490     if jilu["problem"] == 4 and (score >= 0 or not pd):
1491         return True
1492     for c in pd:
1493         if jilu["extra"] < 2 or not ysjc(zt, (q,)):
1494             return False
1495         zhixing(zt, "/measure", q, c)
1496     return True
1497
1498
1499 def zxsj(zt, event):
1500     jilu = zt["ledger"]
1501     unknown = pindao(jilu, ("unknown",))
1502     if not zx(zt, event):
1503         return False
1504     if event["kind"] == "search":
1505         return plcl(zt, unknown) and slcl(zt)
1506     c = event["channel"]
1507     while c in pindao(jilu, ("found",)):
1508         if jilu["extra"] < 2:
1509             return False
1510         next = yddz(
1511             zt["planner"],
1512             jilu["channels"][c]["source"],
1513             c,
1514             zt["position"],

```

```

1515         zt["channel"],
1516         jilu["extra"],
1517         event.get("target"),
1518     )
1519     if not ysjc(zt, next["points"]) or not zx(zt, next):
1520         return False
1521     return slcl(zt)
1522
1523
1524 def zx(zt, event):
1525     jilu = zt["ledger"]
1526     if event["kind"] == "optical_chain":
1527         if jilu["extra"] < len(event["points"]) or not ysjc(zt, event["points"]):
1528             return False
1529         for q in event["points"]:
1530             if event["channel"] not in pindao(jilu, ("found",)):
1531                 return True
1532             zhixing(zt, "/clear", q, event["channel"])
1533         if event["channel"] in pindao(jilu, ("found",)):
1534             raise RuntimeError("Optical route exhausted without clearing the source")
1535         return True
1536     if event["kind"] == "search":
1537         q = event["points"][0]
1538         for c in sorted(
1539             pindao(jilu, ("unknown",)), key=lambda c: (c != zt["channel"], c)
1540         ):
1541             if c not in pindao(jilu, ("unknown",)) or q in jilu["channels"][c]["radio"]:
1542                 continue
1543             if jilu["extra"] < 1 or not ysjc(zt, (q,)):
1544                 return False
1545             zhixing(zt, "/measure", q, c)
1546         return True
1547     if event["kind"] == "clear":
1548         zhixing(zt, "/clear", event["points"][0], event["channel"])
1549         return True
1550     yuan = jilu["channels"][event["channel"]]["source"]
1551     pos = yuan["anchor_point"]
1552     a, b = event["points"]
1553     first = zhixing(zt, "/measure", a, event["channel"])
1554     if first["measure_result"] == "no_signal" and zt["problem"] == 4:
1555         second = zhixing(zt, "/measure", b, event["channel"])
1556         if second["measure_result"] == "no_signal":
1557             syx(yuan, pos, a, b)
1558     return True
1559
1560
1561 def dzqcy(zt, c):
1562     jilu = zt["ledger"]
1563     yuan = jilu["channels"][c]["source"]
1564     order = [(i, yuan["points"][i]) for i in sorted(yuan["cells"])]
1565     for i, q in order:
1566         if c not in pindao(jilu, ("found",)):
1567             return
1568         if i in yuan["cells"]:
1569             zhixing(zt, "/clear", q, c, pd=False)
1570     if c in pindao(jilu, ("found",)):
1571         raise RuntimeError("Constructive optical cover exhausted without success")
1572
1573
1574 def byqc(zt):
1575     jilu = zt["ledger"]
1576     zt["fallback_used"] = True
1577     upper = sysj(jilu, zt["position"])
1578     if zt["virtual"] + upper >= zt["max_virtual"] - 1:
1579         raise RuntimeError("Remaining virtual time is insufficient")
1580     origin = zt["position"]
1581     known = sorted(
1582         pindao(jilu, ("found",)),

```

```

1583         key=lambda c: lxsj(gxpx(jilu["channels"][c]["source"], origin), origin, origin),
1584     )
1585     for c in known:
1586         dzqcy(zt, c)
1587     for q in g.sswd():
1588         for c in sorted(
1589             pindao(jilu, ("unknown",)), key=lambda c: (c != zt["channel"], c)
1590         ):
1591             if (
1592                 c not in pindao(jilu, ("unknown",))
1593                 or q not in jilu["channels"][c]["pending"]
1594             ):
1595                 continue
1596             zhixing(zt, "/measure", q, c, pd=False)
1597             if c in pindao(jilu, ("found",)):
1598                 dzqcy(zt, c)
1599             if qbqc(jilu):
1600                 break
1601     for c in sorted(pindao(jilu, ("unknown",))):
1602         pdwy(jilu, c)
1603     if not qbqc(jilu):
1604         raise RuntimeError(
1605             "Full fallback exhausted without a valid 10-16 source completion"
1606         )
1607
1608
1609 def yunxing(zt):
1610     jiekou = zt["io"]
1611     jilu = zt["ledger"]
1612     huifu = jiekou["call"]("/enter")
1613     zt["entered"] = True
1614     zt["virtual"] = huifu["virtual_time_s"]
1615     zt["max_virtual"] = huifu["max_virtual_duration_s"]
1616     zt["deadline"] = time.monotonic() + huifu["remaining_real_duration_s"]
1617     jiekou["deadline"] = zt["deadline"]
1618     while not qbqc(jilu):
1619         if jilu["extra"] < 2:
1620             byqc(zt)
1621             break
1622         if time.monotonic() + syys(zt) * zt["request_seconds"] + 30 >= zt["deadline"]:
1623             byqc(zt)
1624             break
1625         sslx = gxlx(zt["search_plan"], jilu, zt["position"])
1626         event = xzdz(zt["planner"], jilu, zt["position"], zt["channel"], sslx)
1627         if event is None or not ysjc(zt, event["points"]):
1628             byqc(zt)
1629             break
1630         if not zxsj(zt, event):
1631             byqc(zt)
1632             break
1633     zt["completed"] = qbqc(jilu)
1634     tuichu(zt)
1635     return huizong(zt)
1636
1637
1638 def tuichu(zt):
1639     jiekou = zt["io"]
1640     if not zt["entered"] or zt["exit_attempted"]:
1641         return
1642     if jiekou["pending"] is not None:
1643         raise RuntimeError("Cannot exit while earlier request is unresolved")
1644     zt["exit_attempted"] = True
1645     jiekou["call"]("/exit")
1646     zt["exit_confirmed"] = True
1647
1648
1649 def huizong(zt):
1650     jilu = zt["ledger"]

```

```

1651     cleared = sorted(pindao(jilu, ("cleared",)))
1652     return dict(
1653         problem=zt["problem"],
1654         completed=zt["completed"],
1655         exit_confirmed=zt["exit_confirmed"],
1656         virtual_time_s=zt["virtual"],
1657         moving_distance_m=zt["travel"],
1658         cleared_count=len(cleared),
1659         average_clear_time_s=zt["virtual"] / len(cleared) if cleared else None,
1660         cleared_channels=cleared,
1661         unknown_channels=sorted(pindao(jilu, ("unknown",))),
1662         pending_channels=sorted(pindao(jilu, ("found",))),
1663         measures=zt["measures"],
1664         clear_attempts=zt["clear_attempts"],
1665         switches=zt["switches"],
1666         fallback_used=zt["fallback_used"],
1667     )

```

paper/build.py

```

1  """ 编译论文、完整程序附录和 AI 使用详情。 """
2
3  import argparse
4  from pathlib import Path as lj
5  import shutil
6  import subprocess
7
8  from generate_appendix import shengcheng
9  from make_formal_tables import generate as biaoge
10
11
12 def main():
13     parser = argparse.ArgumentParser(description=__doc__)
14     parser.add_argument("--engine", default="xelatex")
15     parser.add_argument("--format", help=" 预编译的 XeLaTeX 格式文件")
16     args = parser.parse_args()
17     paper = lj(__file__).resolve().parent
18     root = paper.parent
19     build = paper / ".build"
20     build.mkdir(exist_ok=True)
21     (root / "submission").mkdir(exist_ok=True)
22     engine = shutil.which(args.engine)
23     if engine is None:
24         raise SystemExit(
25             " 请安装含 XeLaTeX、ctex、fvextra 和 Fandol 字体的 TeX Live 或 MiKTeX。 "
26         )
27     biaoge(root)
28     shengcheng()
29     cmd = [engine]
30     if args.format:
31         cmd += [f"-fmt={lj(args.format).resolve()}", "-progname=xelatex"]
32     cmd += ["-interaction=nonstopmode", "-halt-on-error", "-output-directory=.build"]
33     for name, target in (
34         ("main", "submission/论文.pdf"),
35         ("ai-usage", "AI 工具使用详情.pdf"),
36     ):
37         for _ in range(2):
38             with (build / f"{name}-compile.log").open("w", encoding="utf-8") as output:
39                 subprocess.run(
40                     cmd + [name + ".tex"],
41                     cwd=paper,
42                     stdout=output,
43                     stderr=subprocess.STDOUT,
44                     check=True,
45                 )
46     shutil.copyfile(build / f"{name}.pdf", root / target)
47     print(f" 已生成 {target}")
48

```

```

49
50 if __name__ == "__main__":
51     main()

```

paper/generate_appendix.py

```

1  """ 生成论文中的完整程序附录。"""
2
3  from pathlib import Path as lj
4
5  root = lj(__file__).resolve().parents[1]
6
7
8  def shengcheng():
9      files = [
10         root / n
11         for n in (
12             "run_robot.py",
13             "solve_questions.py",
14             "start_robot.ps1",
15             "prepare_submission.py",
16         )
17     ]
18     for folder in ("src/jammer_solver", "paper"):
19         files.extend(sorted((root / folder).glob("*.py")))
20     lines = [
21         r"\section*{附录：完整程序清单}",
22         " 以下列出求解、运行、绘图与论文编译所用的完整程序。支撑材料保留相同目录。",
23         r"\begin{group}\linespread{1.0}\selectfont",
24     ]
25     for p in files:
26         name = p.relative_to(root).as_posix()
27         lines.append(r"\subsection*{\texttt{\detokenize{" + name + "}}}")
28         if p.stat().st_size:
29             lines.append(
30                 r"\VerbatimInput[fontsize=\font{8.5}{10.5}\selectfont,"
31                 r"breaklines=true,breakanywhere=true,numbers=left,numbersep=4pt,"
32                 r"xleftmargin=17pt,tabsize=4]{\RepoRoot/" + name + "}"
33             )
34         else:
35             lines.append(" 该文件为空，用于标记 Python 包。")
36     lines.append(r"\end{group}")
37     (root / "paper/appendix_sources.tex").write_text(
38         "\n".join(lines) + "\n", encoding="utf-8"
39     )
40
41
42 if __name__ == "__main__":
43     shengcheng()

```

paper/make_figures.py

```

1  from pathlib import Path
2  import json
3  import math
4  import sys
5  import os
6
7  os.environ.setdefault("MPLCONFIGDIR", "/tmp/mcm-paper-matplotlib")
8  import numpy as np
9  import matplotlib
10
11  matplotlib.use("Agg")
12  import matplotlib.pyplot as plt
13  from matplotlib import font_manager
14  from matplotlib.patches import Circle, Polygon, FancyBboxPatch
15
16  ROOT = Path(__file__).resolve().parents[1]

```

```

17 sys.path.insert(0, str(ROOT / "src"))
18 from jammer_solver import solver as sf, exact_geometry as g
19 from jammer_solver.questions import jdyh, jhqy
20
21 PAPER = ROOT / "paper"
22 DATA = json.loads((PAPER / "results.json").read_text(encoding="utf-8"))
23 font_candidates = [
24     os.environ.get("MCM_PAPER_FONT"),
25     "/usr/share/fonts/truetype/droid/DroidSansFallbackFull.ttf",
26     "C:/Windows/Fonts/simhei.ttf",
27     "/System/Library/Fonts/PingFang.ttc",
28 ]
29 font_family = None
30 for candidate in font_candidates:
31     if candidate and Path(candidate).is_file():
32         font_manager.fontManager.addfont(candidate)
33         font_family = font_manager.FontProperties(fname=candidate).get_name()
34         break
35 if font_family is None:
36     raise SystemExit(
37         "Set MCM_PAPER_FONT to an installed Chinese TrueType font before regenerating figures."
38     )
39 plt.rcParams.update(
40     {
41         "font.family": [font_family, "DejaVu Sans"],
42         "mathtext.fontset": "dejavuserif",
43         "font.size": 10,
44         "axes.labelsize": 10,
45         "legend.fontsize": 9,
46         "axes.titlesize": 11,
47         "axes.unicode_minus": False,
48         "lines.linewidth": 2,
49         "axes.spines.top": False,
50         "axes.spines.right": False,
51         "savefig.dpi": 320,
52         "pdf.fonttype": 42,
53     }
54 )
55 BLUE = "#0072B2"
56 RED = "#D55E00"
57 GREEN = "#009E73"
58 PURPLE = "#7B61A8"
59 GRAY = "#555555"
60
61
62 def save(fig, name):
63     fig.savefig(PAPER / "figures" / f"{name}.pdf", bbox_inches="tight", pad_inches=0.04)
64     plt.close(fig)
65
66
67 def equal(ax, xlabel=" 东向坐标 / m", ylabel=" 北向坐标 / m"):
68     ax.set_aspect("equal")
69     ax.set_xlabel(xlabel)
70     ax.set_ylabel(ylabel)
71     ax.grid(alpha=0.18)
72
73
74 fig, ax = plt.subplots(figsize=(6.3, 3.2))
75 ax.set_xlim(0, 10)
76 ax.set_ylim(0, 5)
77 ax.axis("off")
78 boxes = [
79     (0.1, 3.2, 2.65, 1.0, " 实际观测\n 方向 / 近距 / 阴性", BLUE),
80     (3.6, 3.2, 2.65, 1.0, " 证据账本\n 频道状态与可行域", BLUE),
81     (7.1, 3.2, 2.65, 1.0, " 完成条件\n 实际清除 + 排除未知", GREEN),
82     (0.1, 0.55, 2.65, 1.15, " 覆盖搜索\n Q3: 7 站 / Q4: 21 站", GRAY),
83     (3.6, 0.55, 2.65, 1.15, " 联合任务排序\n 移动 + 测向 + 清除", BLUE),
84     (7.1, 0.55, 2.65, 1.15, " 执行下一动作\n 射频 / 完整光学链", RED),

```

```

85 ]
86 for x, y, w, h, t, c in boxes:
87     ax.add_patch(
88         FancyBboxPatch(
89             (x, y),
90             w,
91             h,
92             boxstyle="round,pad=.06",
93             linewidth=1.5,
94             edgecolor=c,
95             facecolor="white",
96         )
97     )
98     ax.text(x + w / 2, y + h / 2, t, ha="center", va="center", fontsize=10)
99 for a, b in [
100     ((2.8, 3.7), (3.5, 3.7)),
101     ((6.3, 3.7), (7, 3.7)),
102     ((2.8, 1.1), (3.5, 1.1)),
103     ((6.3, 1.1), (7, 1.1)),
104     ((4.9, 3.1), (4.9, 1.8)),
105     ((8.5, 1.8), (8.5, 2.4)),
106     ((8.5, 2.4), (1.4, 2.4)),
107     ((1.4, 2.4), (1.4, 3.1)),
108 ]:
109     ax.annotate("", b, a, arrowprops=dict(arrowstyle="->", lw=1.5, color=GRAY))
110 ax.text(5, 4.65, " 评分决定 “先做什么”；证书决定 “能否完成” ", ha="center", fontsize=11)
111 save(fig, "flow")
112
113 fig, axs = plt.subplots(1, 2, figsize=(6.3, 2.8), layout="constrained")
114 obs = [dict(x=0, y=0, bearing_deg=45), dict(x=100, y=0, bearing_deg=135)]
115 region = jhqy(obs)
116 poly = np.array(region["vertices"])
117 ax = axs[0]
118 ax.add_patch(Polygon(poly, facecolor="#DCECF5", edgecolor=BLUE, lw=2))
119 a, b = np.array(region["farthest_pair"])
120 ax.plot([a[0], b[0]], [a[1], b[1]], "o--", color=RED, ms=4)
121 ax.set_xlim(47.5, 52.5)
122 ax.set_ylim(47.5, 52.5)
123 equal(ax)
124 ax.set_title("(a) 两次观测的交会区域")
125 ax.text(50, 47.8, "D = 3.5096 m", ha="center", fontsize=9)
126 ax = axs[1]
127 tri = np.array([[ -0.5, 0], [0.5, 0], [0, np.sqrt(3) / 2]])
128 ax.add_patch(Polygon(tri, fill=False, edgecolor="black", lw=2))
129 ax.add_patch(Circle((0, 0), 0.5, fill=False, ls="--", color=RED, lw=2))
130 ax.add_patch(
131     Circle((0, np.sqrt(3) / 6), 1 / np.sqrt(3), fill=False, ls="-. ", color=BLUE, lw=2)
132 )
133 ax.scatter(*tri.T, color="black", s=20)
134 ax.set_xlim(-0.8, 0.8)
135 ax.set_ylim(-0.65, 1)
136 equal(ax, "x / d", "y / d")
137 ax.set_title("(b) 同直径圆的反例")
138 ax.plot([], [], "--", color=RED, label=" 半径 d/2")
139 ax.plot([], [], "-.", color=BLUE, label=r" 半径 $d/\sqrt{3}$")
140 ax.legend(loc="lower center", fontsize=8)
141 save(fig, "q1_geometry")
142
143 fig, axs = plt.subplots(1, 2, figsize=(6.3, 2.85), layout="constrained")
144 a = np.linspace(0, 1100, 500)
145 b = np.linspace(-1100, 1100, 600)
146 A, B = np.meshgrid(a, b)
147 eps = math.radians(1.005)
148 mask = A * A + B * B <= 1000**2
149 for s in [-1, 1]:
150     mask &= (A - 1000 * math.cos(eps)) ** 2 + (
151         B - s * 1000 * math.sin(eps)
152     ) ** 2 <= 1000**2

```

```

153 axs[0].contourf(A, B, mask.astype(float), levels=[0.5, 1.5], colors=["#DCECF5"])
154 axs[0].contour(A, B, mask.astype(float), levels=[0.5], colors=[BLUE], linewidths=1.5)
155 p = DATA["q2_minimum"]
156 axs[0].plot(p["a_m"], p["b_m"], "*", ms=10, color=RED)
157 axs[0].plot(500, 840, "s", ms=4, color=GREEN)
158 axs[0].set_title("(a) 保证接收的候选区域")
159 equal(axs[0], " 前向位移 a / m", " 侧向位移 b / m")
160 rs = np.linspace(500, 999, 50)
161 vals = [jdyh(float(r))["diameter_upper_m"] for r in rs]
162 axs[1].plot(rs, vals, color=BLUE)
163 axs[1].axhline(163, ls="--", color=RED, lw=1.5)
164 axs[1].plot(p["movement_m"], p["diameter_upper_m"], "o", color=RED)
165 axs[1].set_xlabel(" 移动预算 \u03c1 / m")
166 axs[1].set_ylabel(" 解析直径上界 / m")
167 axs[1].set_title("(b) 移动与定位精度的折中")
168 axs[1].grid(alpha=0.18)
169 save(fig, "q2_design")
170
171 fig, axs = plt.subplots(1, 2, figsize=(6.3, 3.1), layout="constrained")
172 for ax, problem in zip(axs, [3, 4]):
173     q = np.array(sf.sscd(problem))
174     ax.add_patch(Circle((0, 0), 1800, fill=False, color="black", lw=1.6))
175     if problem == 3:
176         for p in q:
177             ax.add_patch(Circle(p, 1000, fill=False, color=BLUE, lw=0.65, alpha=0.45))
178     else:
179         ax.plot(*q[1:9].T, "o", color=BLUE, ms=4)
180         ax.plot(*q[9:].T, "s", color=RED, ms=4)
181         ax.plot(q[:, 0], q[:, 1], "o", color=BLUE, ms=3.5)
182         ax.plot(0, 0, "*", color="black", ms=9)
183         ax.set_xlim(-2400, 2400)
184         ax.set_ylim(-2400, 2400)
185         equal(ax)
186         ax.set_xticks([-1800, 0, 1800])
187         ax.set_yticks([-1800, 0, 1800])
188         ax.set_title(f" 问题 {problem}: {len(q)} 个搜索站")
189 save(fig, "coverage_layouts")
190
191 fig, axs = plt.subplots(1, 2, figsize=(6.3, 2.95), layout="constrained")
192 ax = axs[0]
193 source = np.array([0.0, 0.0])
194 witness = np.array([[-450, -300], [500, -200], [150, 550]])
195 ax.add_patch(Polygon(witness, facecolor="#DCECF5", edgecolor=BLUE, lw=2))
196 ax.scatter(*witness.T, color=BLUE, s=35)
197 ax.plot(0, 0, "*", ms=12, color=RED)
198 ax.fill_between([-650, 700], -600, 600, color=GREEN, alpha=0.05)
199 ax.axvline(0, color=GRAY, ls="--", lw=1.5)
200 ax.annotate(
201     " 任意闭半平面\mathcal{H} 至少包含一个证据点",
202     (20, 10),
203     (70, -530),
204     arrowprops=dict(arrowstyle="->", lw=1.3),
205     fontsize=9,
206 )
207 ax.set_xlim(-650, 700)
208 ax.set_ylim(-650, 750)
209 equal(ax)
210 ax.set_title("(a) 源位于近邻测点凸包内")
211 ax = axs[1]
212 o = (0, 0)
213 a = (750, -200)
214 b = (750, 200)
215 ax.add_patch(
216     Polygon(
217         [a, (1250, -1000 / 3), (1250, 1000 / 3), b],
218         facecolor="#F5DFD4",
219         edgecolor=RED,
220         lw=2,

```

```

221     )
222 )
223 ax.plot([0, 1300], [0, 0], ":", color=GRAY)
224 ax.plot(0, 0, "*", ms=11, color=GREEN)
225 ax.plot([750, 750], [-200, 200], "s", color=RED)
226 ax.text(60, 60, " 实际阳性锚点", fontsize=9)
227 ax.text(1020, -80, " 可排除部分", ha="center", fontsize=9)
228 ax.text(780, 360, " 双阴性测点", fontsize=9)
229 ax.set_xlim(-100, 1350)
230 ax.set_ylim(-500, 500)
231 equal(ax)
232 ax.set_title("(b) 双阴性收缩的几何条件")
233 save(fig, "directional_geometry")
234
235 from fractions import Fraction
236
237 snapshot = json.loads((PAPER / "figure-data.json").read_text(encoding="utf-8"))
238 poly = tuple(tuple(Fraction(x) for x in p) for p in snapshot["polygon"])
239 points = [g.point(p) for p in snapshot["points"]]
240 origin = np.array((tuple((float(x) for x in points[0]))))
241 axis = np.array((tuple((float(x) for x in points[-1])))) - origin
242 axis /= np.linalg.norm(axis)
243 normal = np.array([-axis[1], axis[0]])
244
245
246 def local(point):
247     delta = np.array((tuple((float(x) for x in point)))) - origin
248     return np.array([delta @ axis, delta @ normal])
249
250
251 fig, ax = plt.subplots(figsize=(6.3, 2.35), layout="constrained")
252 colors = [BLUE, RED, GREEN, PURPLE]
253 for i, q in enumerate(points):
254     part = poly
255     for other in points:
256         if q != other:
257             part = g.clip(
258                 part,
259                 ((other[0] - q[0], other[1] - q[1])),
260                 (
261                     (other[0] * other[0] + other[1] * other[1])
262                     - (q[0] * q[0] + q[1] * q[1])
263                 )
264                 / 2,
265             )
266     color = colors[i % len(colors)]
267     qf = local(q)
268     ax.add_patch(Circle(qf, 19.8, fill=False, ls="--", color=color, lw=1.6))
269     if part:
270         ax.add_patch(
271             Polygon(
272                 [local(v) for v in part], facecolor=color, edgecolor=color, alpha=0.23
273             )
274         )
275     ax.plot(*qf, "o", color=color, ms=4)
276     ax.annotate(str(i + 1), qf, xytext=(5, 4), textcoords="offset points", fontsize=9)
277 qs = np.array([local(q) for q in points])
278 ax.plot(qs[:, 0], qs[:, 1], "-", color=GRAY, lw=1.2)
279 ax.add_patch(Polygon([local(v) for v in poly], fill=False, color="black", lw=2))
280 ax.autoscale_view()
281 equal(ax, " 沿链轴坐标 / m", " 横向坐标 / m")
282 ax.set_title(" 实际决策时的完整可行域与光学覆盖链")
283 save(fig, "optical_chain")
284
285 fig, axs = plt.subplots(1, 2, figsize=(6.3, 2.85), layout="constrained")
286 for p, c, m in [(3, BLUE, "o"), (4, RED, "s")]:
287     rows = [
288         r

```

```

289         for r in DATA["cases"]
290         if r["name"].startswith(f"q{p}-") and r["category"] == "standard"
291     ]
292     axs[0].scatter(
293         [r["source_count"] for r in rows],
294         [r["virtual_time_s"] for r in rows],
295         c=c,
296         marker=m,
297         s=28,
298         label=f" 问题 {p}",
299     )
300     axs[0].set_xlabel(" 源个数")
301     axs[0].set_ylabel(" 总虚拟时间 / s")
302     axs[0].set_xticks([10, 12, 14, 16])
303     axs[0].legend()
304     axs[0].grid(alpha=0.18)
305     for j, p in enumerate([3, 4]):
306         rows = [
307             r
308             for r in DATA["cases"]
309             if r["name"].startswith(f"q{p}-") and r["category"] == "standard"
310         ]
311         costs = [
312             np.mean([r["moving_distance_m"] / 5 for r in rows]),
313             np.mean([r["counts"]["measure"] * 5 for r in rows]),
314             np.mean([r["counts"]["switch"] for r in rows]),
315             np.mean(
316                 [
317                     r["counts"]["successful_clear"] * 5
318                     + (r["counts"]["clear"] - r["counts"]["successful_clear"]) * 3
319                     for r in rows
320                 ]
321             ),
322         ]
323         base = 0
324         for i, (value, color, label) in enumerate(
325             zip(costs, [BLUE, RED, GREEN, PURPLE], [" 移动", " 测向", " 切频", " 光学与清除"])
326         ):
327             axs[1].bar(
328                 j,
329                 value,
330                 bottom=base,
331                 width=0.58,
332                 color=color,
333                 label=label if j == 0 else None,
334                 hatch=["", "//", "xx", "."][i],
335             )
336             base += value
337     axs[1].set_xticks([0, 1], [" 问题 3", " 问题 4"])
338     axs[1].set_ylabel(" 平均虚拟时间 / s")
339     axs[1].legend(fontsize=8, ncol=2, loc="upper left")
340     axs[1].set_ylim(0, 8200)
341     save(fig, "performance")
342
343     fig, axs = plt.subplots(1, 2, figsize=(6.3, 2.7), layout="constrained")
344     distance = np.linspace(0, 1000, 101)
345     axs[0].plot(distance, distance / 5, "-", color=BLUE)
346     axs[0].set_xlabel(" 移动距离 / m")
347     axs[0].set_ylabel(" 移动时间 / s")
348     axs[0].grid(alpha=0.18)
349     axs[1].bar(
350         [" 测向", " 切频", " 清除成功", " 光学失败"],
351         [5, 1, 5, 3],
352         color=[BLUE, GREEN, RED, PURPLE],
353         hatch=["", "//", "xx", "."],
354     )
355     axs[1].set_ylabel(" 单次设备操作时间 / s")
356     axs[1].set_ylim(0, 6)

```

```

357 save(fig, "task_costs")
358
359 fig, axs = plt.subplots(1, 2, figsize=(6.3, 2.65), layout="constrained")
360 for ax, p in zip(axs, [3, 4]):
361     rows = [r for r in DATA["practice"] if r["problem"] == p]
362     x = [
363         r["reported_source_count"] if p == 3 else r["reported_directional"]
364         for r in rows
365     ]
366     y = [r["reported_virtual_time_s"] for r in rows]
367     ax.scatter(x, y, color=BLUE if p == 3 else RED, s=32)
368     for i, (xx, yy) in enumerate(zip(x, y)):
369         ax.annotate(
370             str(i + 1), (xx, yy), xytext=(5, 4), textcoords="offset points", fontsize=9
371         )
372     ax.set_xlabel(" 截图报告源个数" if p == 3 else " 截图报告定向源个数")
373     ax.set_ylabel(" 截图报告总时间 / s")
374     ax.set_title(f" 问题 {p}: 五次不同案例")
375     ax.grid(alpha=0.18)
376 save(fig, "practice")
377 print("Generated 9 figures in PDF format")

```

paper/make_formal_tables.py

```

1  """Use measured CSV values when present; preserve genuinely empty result cells."""
2
3  import csv
4  from pathlib import Path
5
6  ROOT = Path(__file__).resolve().parents[1]
7
8
9  def escape(value):
10     replacements = {
11         "\\": r"\textbackslash{}",
12         "&": r"&",
13         "%": r"%",
14         "$": r"\$",
15         "#": r"\#",
16         "_": r"\_",
17         "{": r"\{",
18         "}": r"\}",
19         "~": r"\textasciitilde{}",
20         "^": r"\textasciicircum{}",
21     }
22     return "".join(replacements.get(c, c) for c in str(value))
23
24
25  def generate(root=ROOT):
26     with (root / "submission_inputs/formal_results.csv").open(
27         encoding="utf-8-sig", newline=""
28     ) as f:
29         rows = list(csv.DictReader(f))
30         for problem in (3, 4):
31             lines = []
32             for index in (1, 2, 3):
33                 row = next(
34                     r
35                     for r in rows
36                     if r["problem"] == str(problem) and r["test"] == str(index)
37                 )
38                 cells = [
39                     row[key].strip()
40                     for key in (
41                         "case_code",
42                         "cleared_count",
43                         "average_time_s",
44                         "real_time_s",

```

```

45         )
46     ]
47     lines.append(
48         " & ".join(escape(c) if c else r"\blank" for c in cells) + r"\\[10pt]"
49     )
50     lines[-1] += r"\bottomrule"
51     (root / f"paper/formal_q{problem}.tex").write_text(
52         "\n".join(lines) + "\n", encoding="utf-8"
53     )
54
55
56 if __name__ == "__main__":
57     generate()

```

paper/make_tables.py

```

1 from pathlib import Path
2 import json
3
4 PAPER = Path(__file__).resolve().parent
5 data = json.loads((PAPER / "results.json").read_text(encoding="utf-8"))
6
7
8 for problem in (3, 4):
9     rows = [r for r in data["cases"] if r["name"].startswith(f"q{problem}-")]
10    if problem == 3:
11        header = r" 案例 & 源数 & 总时间/s & 平均时间/(s/源) & $M$ & $K_f$"
12        specs = "crrrrr"
13    else:
14        header = r" 案例 & 全向/定向 & 总时间/s & 平均时间/(s/源) & $K_f$"
15        specs = "ccrrr"
16    lines = [
17        r"\begin{table}[H]\centering\caption{问题"
18        + str(problem)
19        + r" 的 15 场离线固定输入结果}\label{tab:q"
20        + str(problem)
21        + "}",
22        r"\small\setlength{\tabcolsep}{7pt}",
23        r"\begin{tabular}{ " + specs + r"}\toprule",
24        header + r"\\midrule",
25    ]
26    for case in rows:
27        n, t, counts = case["source_count"], case["virtual_time_s"], case["counts"]
28        failure = counts["clear"] - counts["successful_clear"]
29        if problem == 3:
30            line = f"{case['label']} & {n} & {t:.2f} & {t / n:.2f} & {counts['measure']} & \\
31            ↪ {failure}"
32        else:
33            d = case["directional_count"]
34            line = f"{case['label']} & {n - d}/{d} & {t:.2f} & {t / n:.2f} & {failure}"
35        lines.append(line + r"\\")
36    lines += [r"\bottomrule\end{tabular}", r"\end{table}"]
37    (PAPER / f"table_q{problem}.tex").write_text(
38        "\n".join(lines) + "\n", encoding="utf-8"
39    )
40
41 lines = [
42     r"\begin{table}[H]\centering\caption{十次演练截图的成绩汇总}\label{tab:practice}",
43     r"\small\setlength{\tabcolsep}{4pt}\begin{tabular}{ccrr}\toprule",
44     r" 题号/次序 & 案例编码 & 报告全向/定向 & 报告个数 & 总时间 (约) /s\\midrule",
45 ]
46 for problem in (3, 4):
47     for i, row in enumerate(
48         [r for r in data["practice"] if r["problem"] == problem], 1
49     ):
50         lines.append(
51             f"{problem}/{i} & {row['case_code']} & "
52             f"{row['reported_omnidirectional']}/{row['reported_directional']} & "

```

```

52         f"{row['reported_source_count']} & {row['reported_virtual_time_s']}" + r"\"
53     )
54 lines += [r"\bottomrule\end{tabular}", r"\end{table}"]
55 (PAPER / "table_practice.tex").write_text("\n".join(lines) + "\n", encoding="utf-8")

```